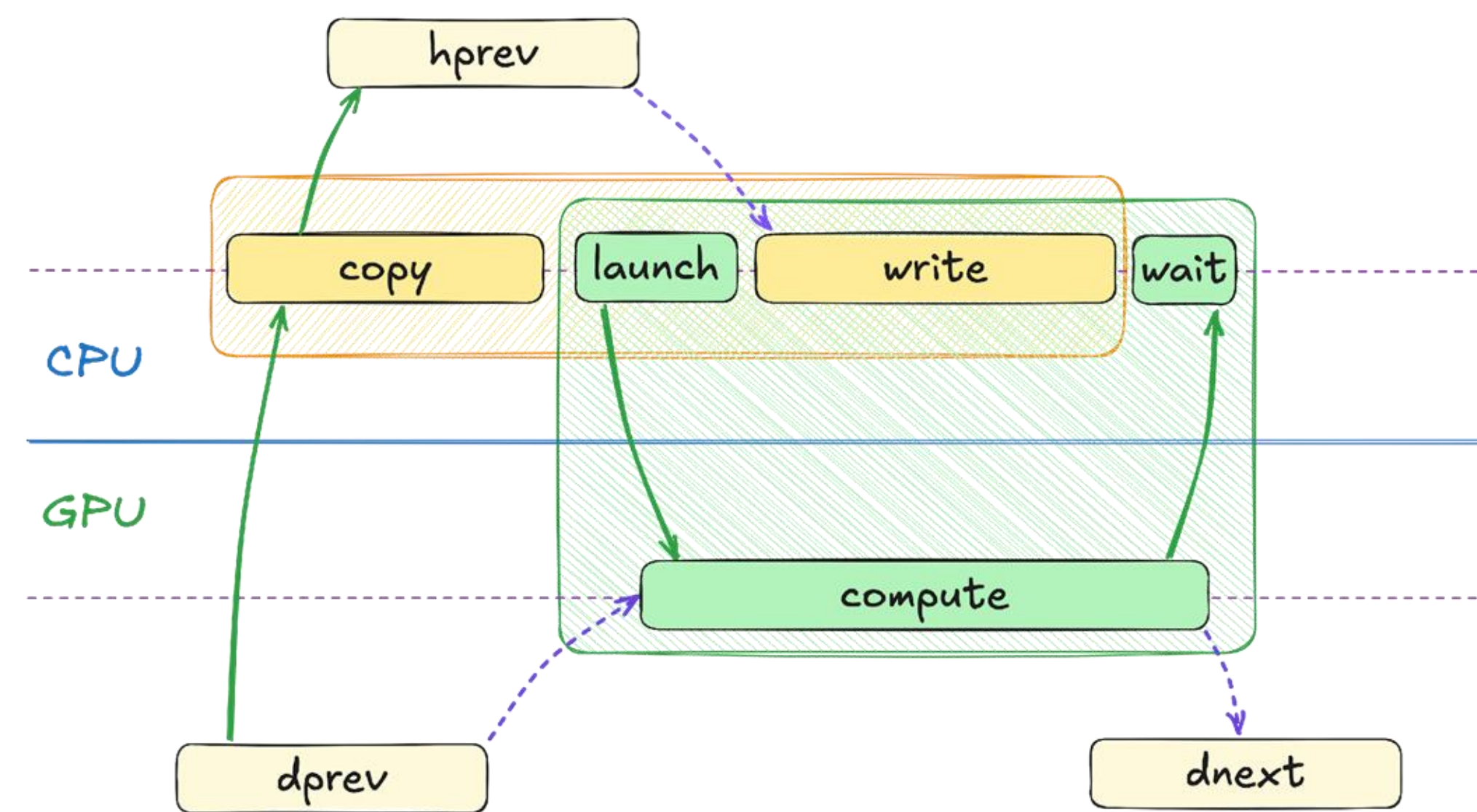


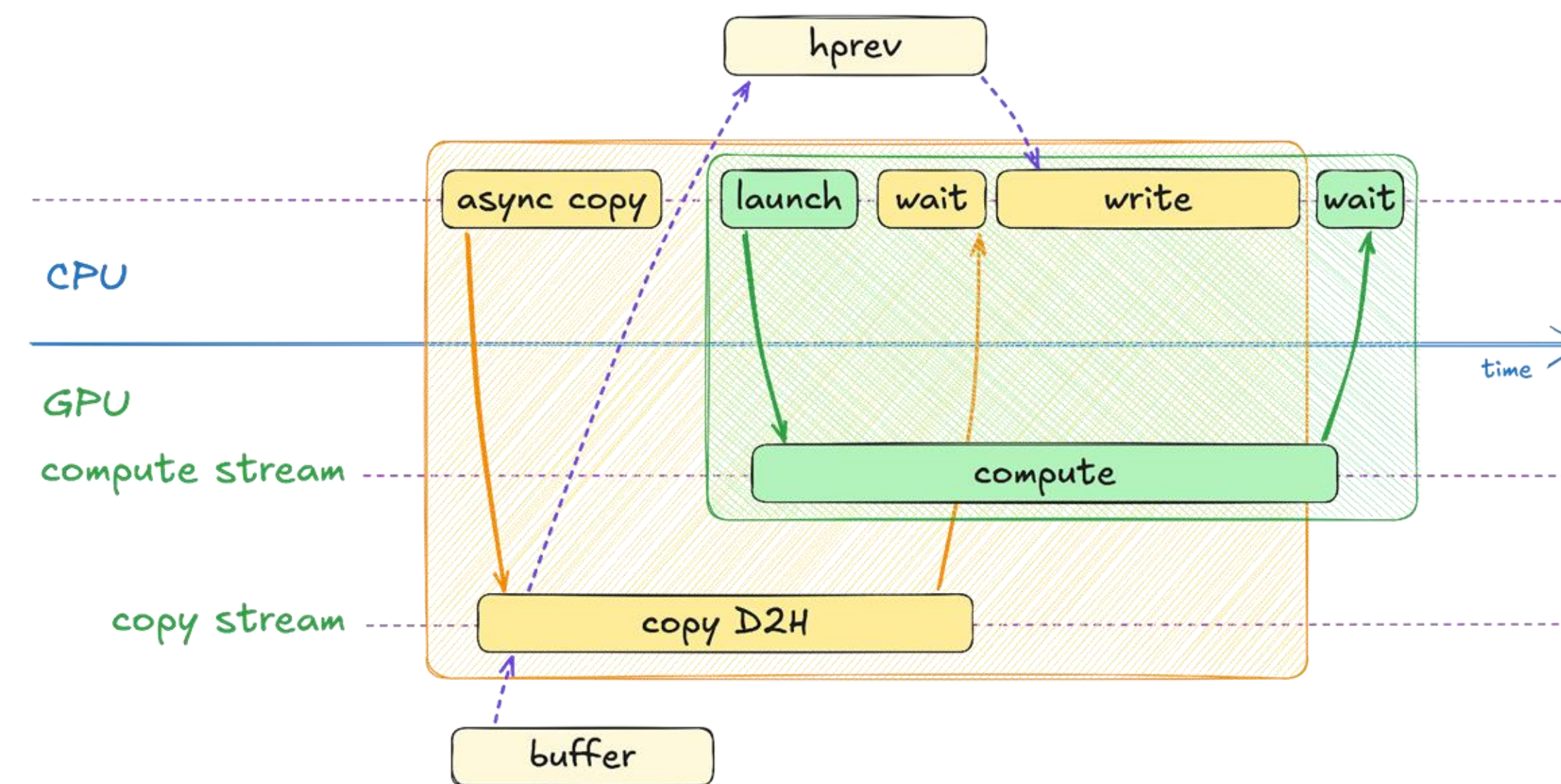
GPU의 잠재력 최대한 활용: 비동기 및 CUDA 스트림

섹션 개요



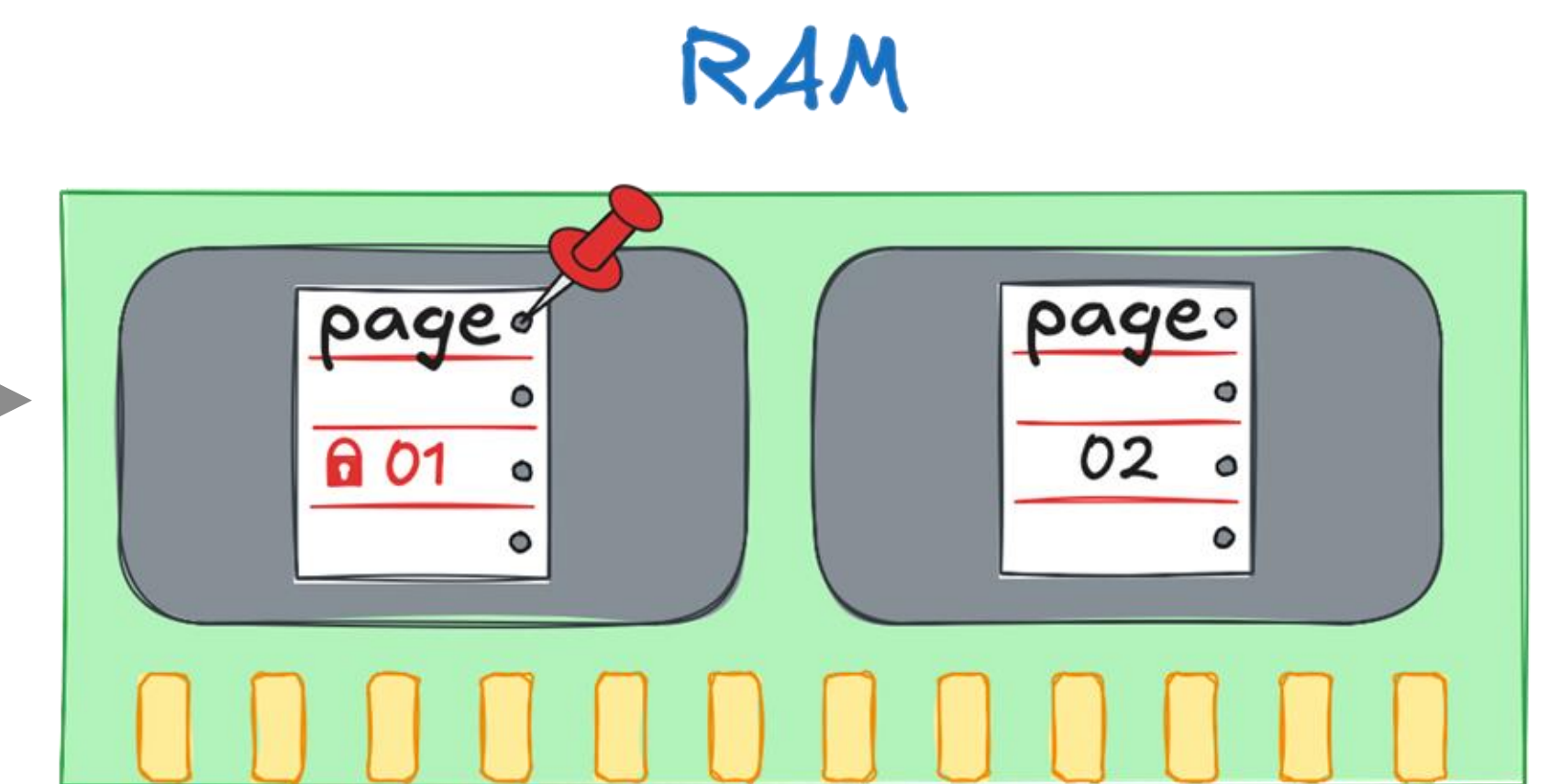
Learn the notion of 2.2 **Asynchrony** as we overlap computation and IO to achieve 2x speedup

2배 속도 향상을 달성하기 위해 계산과 IO를 겹치면서 2.2 비동기의 개념을 알아보세요



Learn the notion of 2.3 **CUDA Streams** as we overlap computation with memory transfers

2.3 CUDA 스트림의 개념을 배우면서 계산과 메모리 전송을 겹치게 합니다.

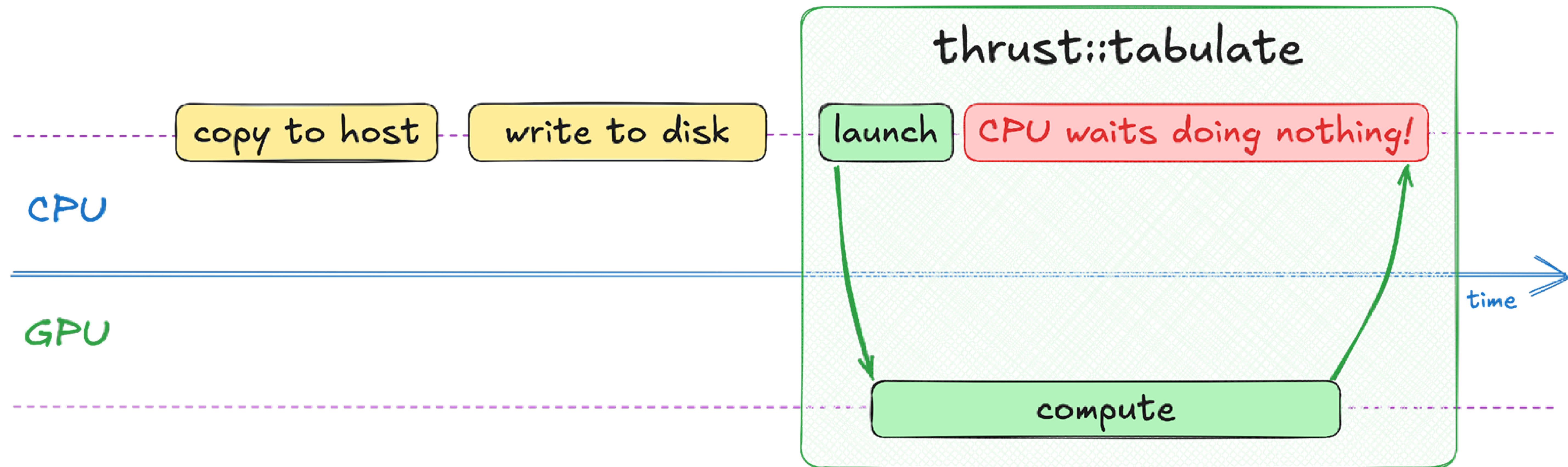


Learn the notion of 2.4 **Pinned Memory** as we make memory transfers asynchronous

메모리 전송을 비동기식으로 만들 때 2.4 Pinned 메모리의 개념을 배우십시오.

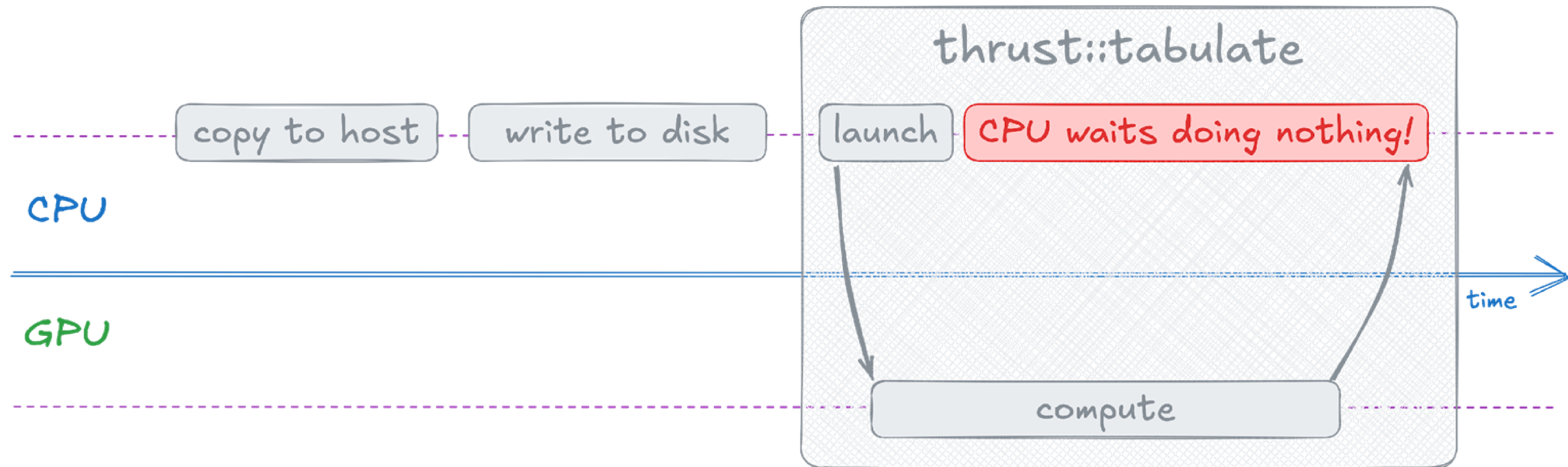
현재 상태 요약

- 이전 데이터를 CPU에 복사하고 디스크에 쓰기
- 이전 상태를 읽고 다음 상태를 쓰는 `thrust::tabulate` 호출



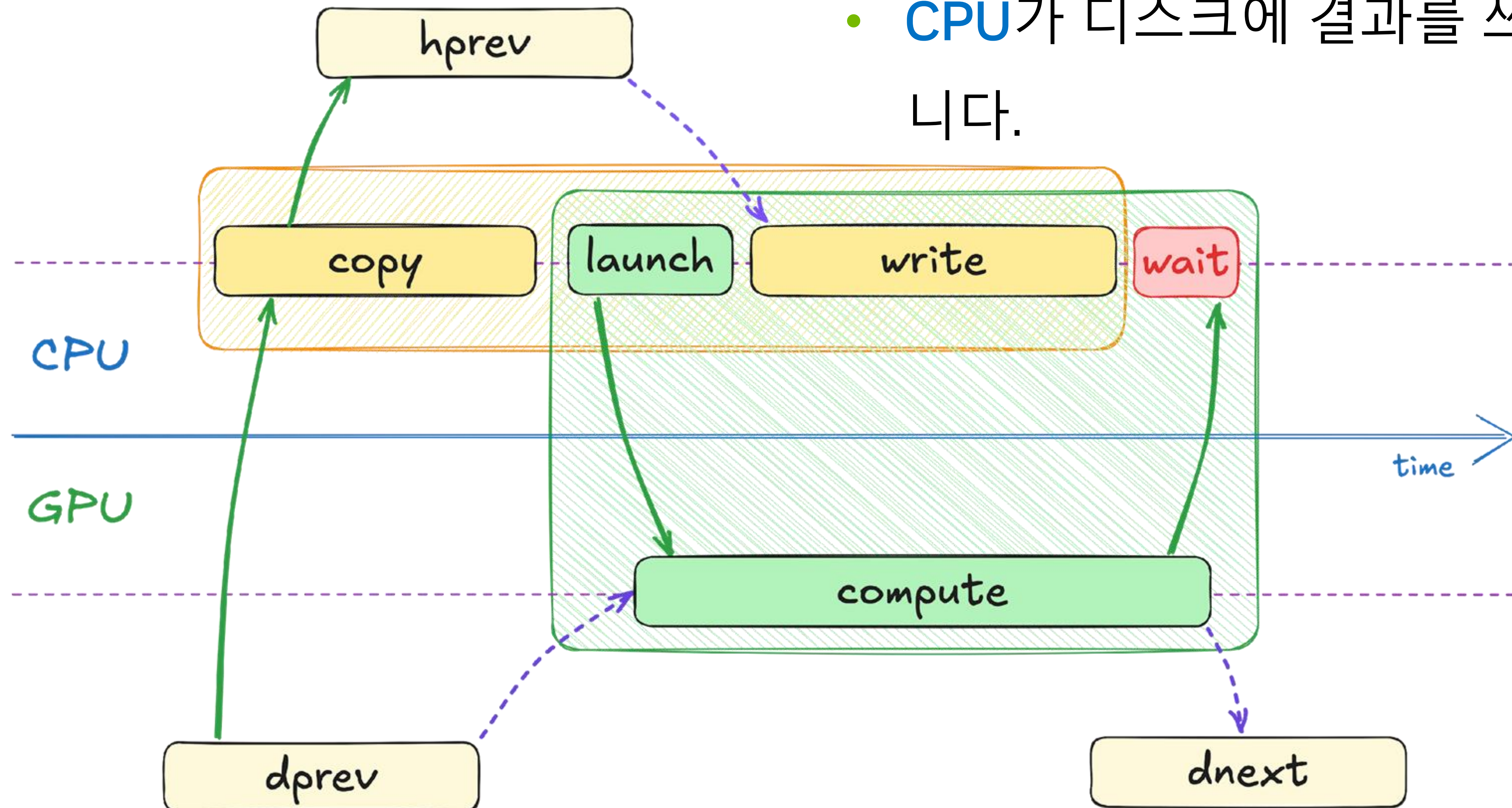
Idle 유틸 리소스

- 내부적으로 Thrust는 GPU에서 작업을 시작하고 완료될 때까지 기다립니다(동기식).
- 효율적인 이기종 프로그램을 작성한다는 것은 **CPU**를 포함한 모든 수단을 활용하는 것을 의미합니다.
- **GPU**가 사용 중인 동안 **CPU**가 할 수 있는 유용한 작업을 찾을 수 있습니까?



데이터 종속성

- 쓰기 단계는 이전 상태의 호스트 복사본을 읽습니다.
- 컴퓨팅 단계가 이전 상태의 디바이스 복사본을 읽습니다.
- 따라서 둘 사이에는 종속성이 없습니다.
- CPU가 디스크에 결과를 쓰는 데 시간을 할애할 수 있음을 의미합니다.



- 그러나 이를 위해서는 Thrust 호출을 실행으로 분할하고 기다려야 합니다

CUB

Sorting

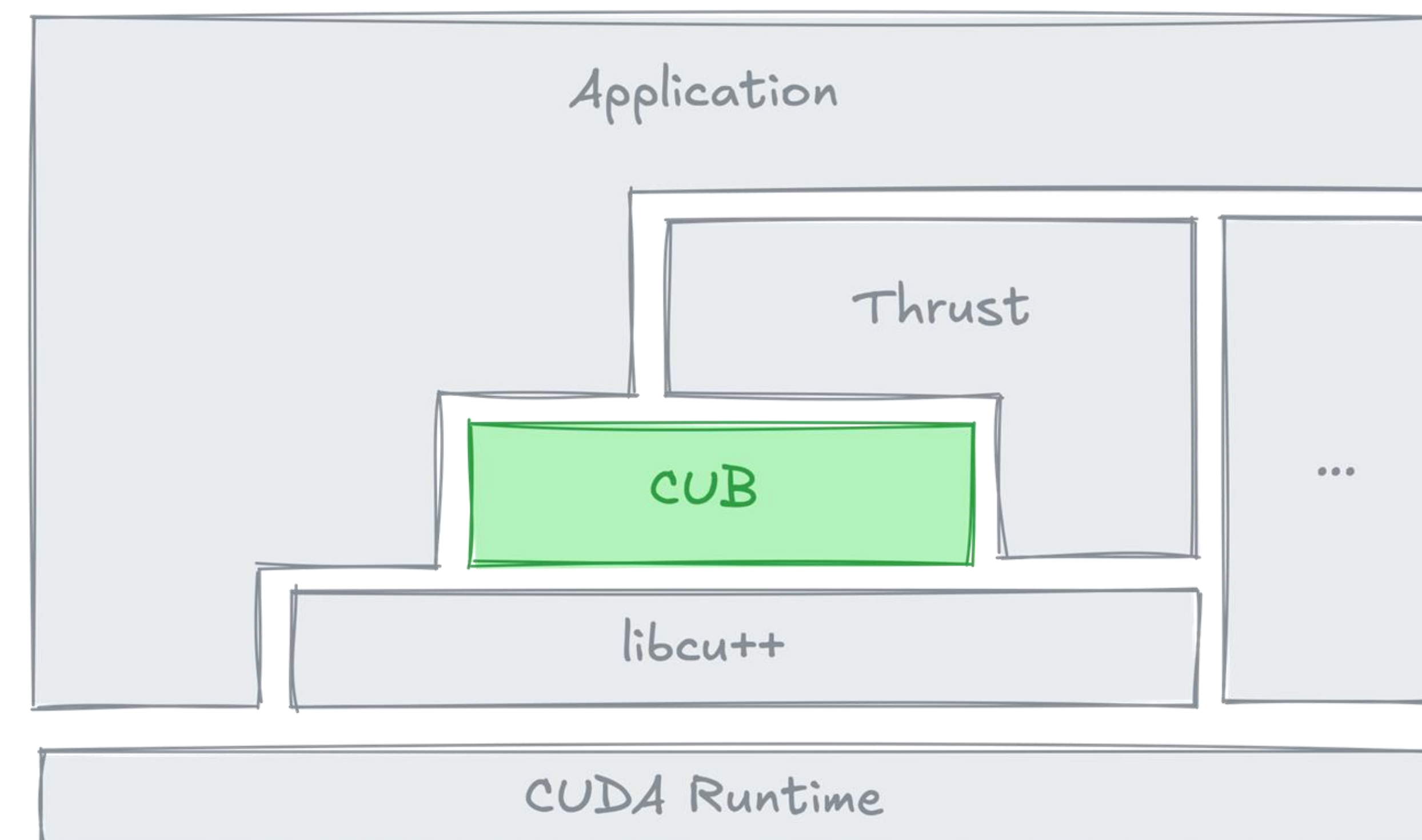
- `cub::DeviceTransform`
- `cub::DeviceRadixSort`
- ...

Types

- `cub::FutureValue`
- `cub::DoubleBuffer`
- ...

Iterators

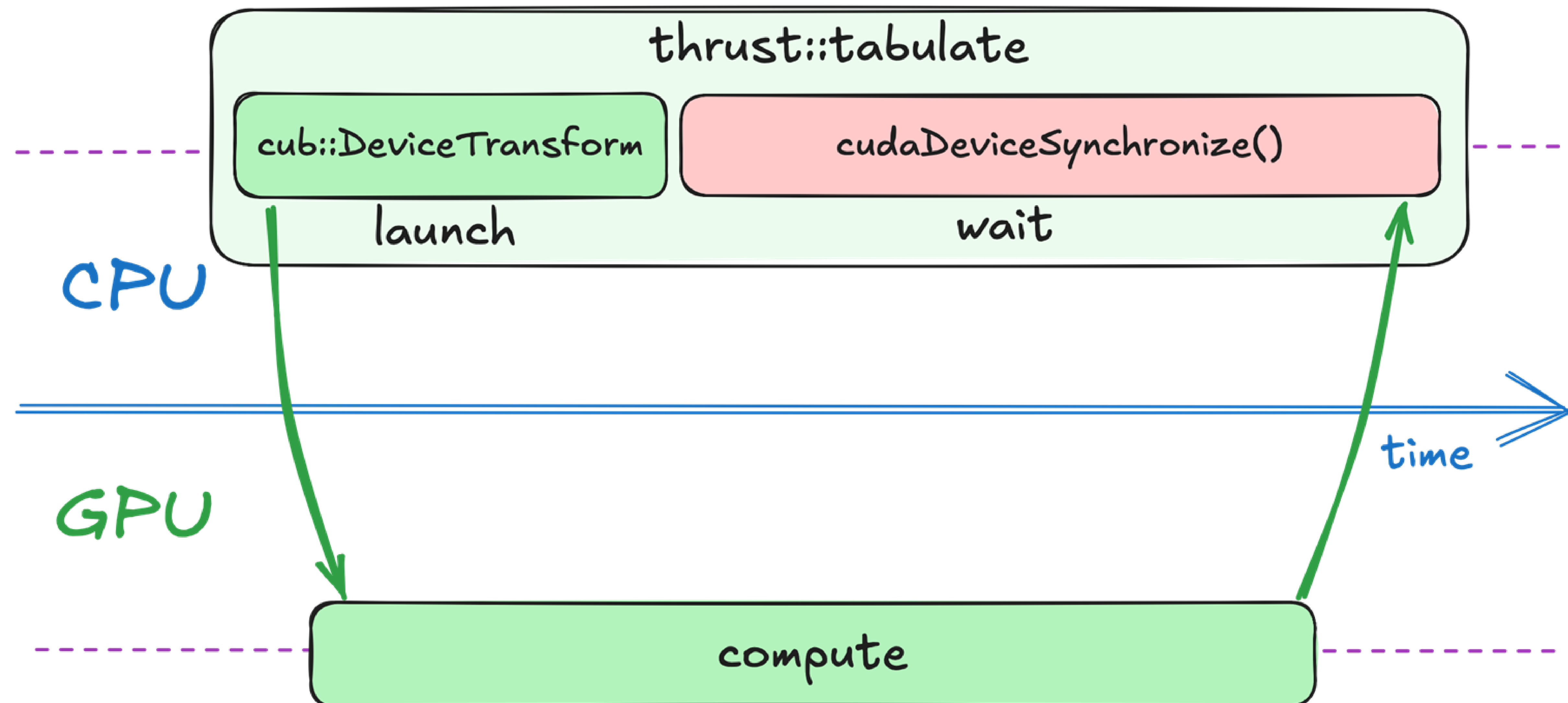
- `cub::CacheModifiedInputIterator`
- `cub::CacheModifiedOutputIterator`
- ...



<https://nvidia.github.io/cccl/cub>

Synchronous vs Asynchronous

- CUB라는 CUDA 코어 라이브러리의 다른 계층에 대한 Thrust 호출
- Thrust와 달리 CUB는 비동기식입니다(제어 흐름은 알고리즘이 완료될 때까지 기다리지 않음).
- GPU의 모든 비동기 작업과 동기화하기 위해 Thrust는 `cudaDeviceSynchronize` 를 사용합니다.



CUB는 비동기입니다.

Thrust

```
auto begin = std::chrono::high_resolution_clock::now();
thrust::tabulate(thrust::device, out.begin(), out.end(), compute);
auto end = std::chrono::high_resolution_clock::now();
```

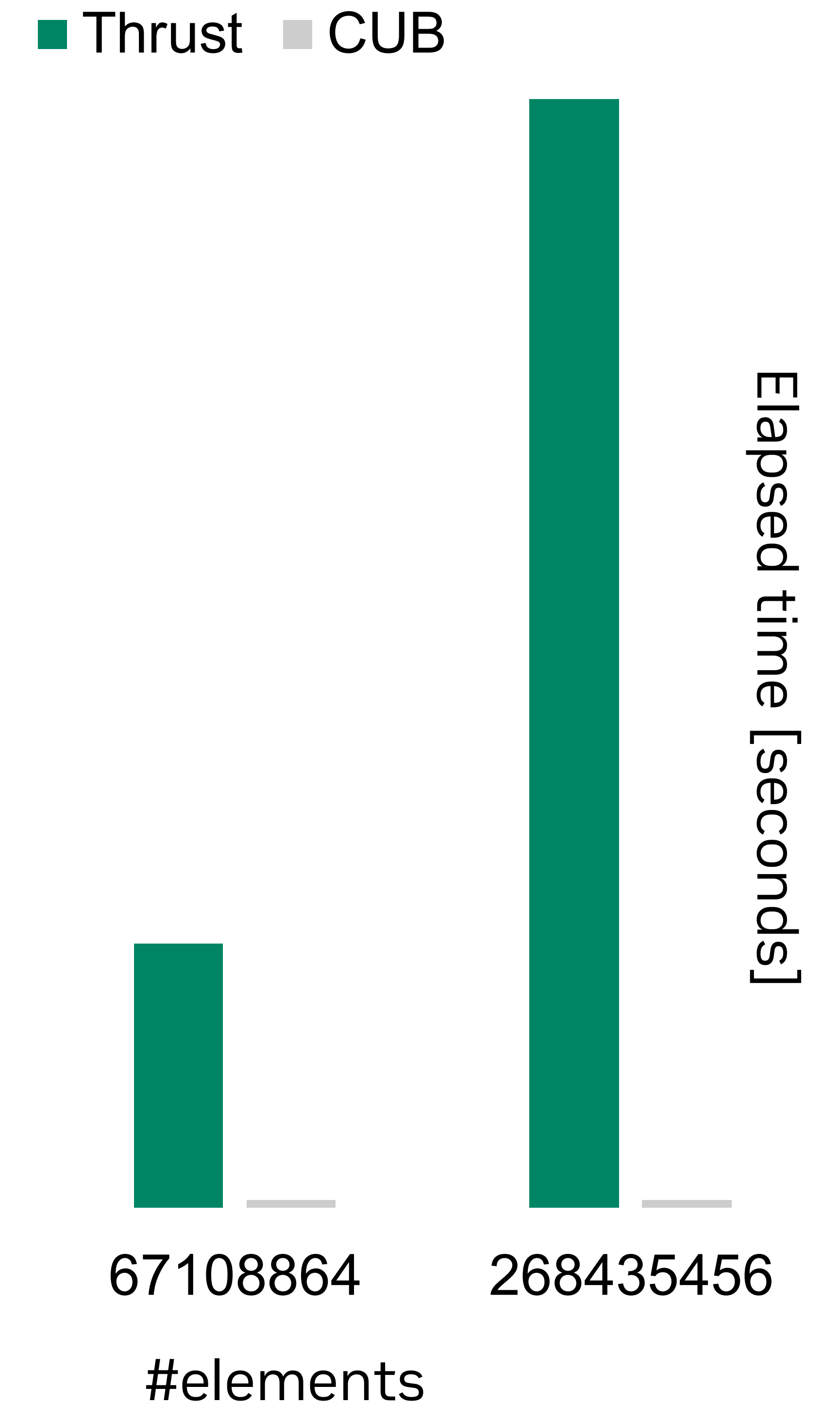
CUB

```
auto begin = std::chrono::high_resolution_clock::now();
auto cell_ids = thrust::make_counting_iterator(0);
cub::DeviceTransform::Transform(cell_ids, out.begin(), num_cells, compute);

auto end = std::chrono::high_resolution_clock::now();
```

- CPU는 다음 명령(종료 시간 기록)을 실행하기 전에 변환이 완료될 때까지 기다리지 않습니다.

- 그렇기 때문에 CUB 경과 시간이 문제 크기에 따라 확장되지 않습니다.



명시적 디바이스 동기화

Thrust

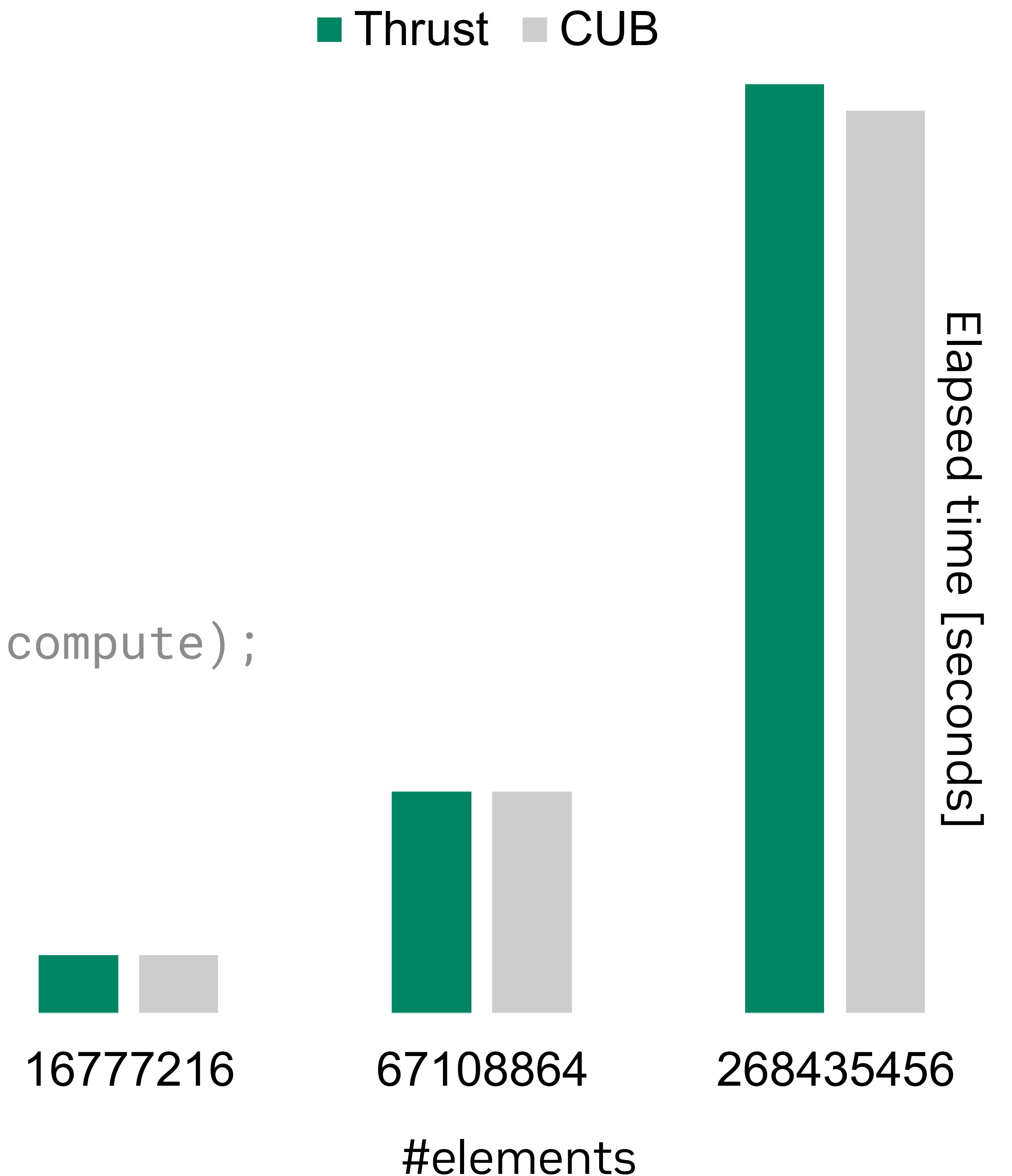
```
auto begin = std::chrono::high_resolution_clock::now();
thrust::tabulate(thrust::device, out.begin(), out.end(), compute);
auto end = std::chrono::high_resolution_clock::now();
```

CUB

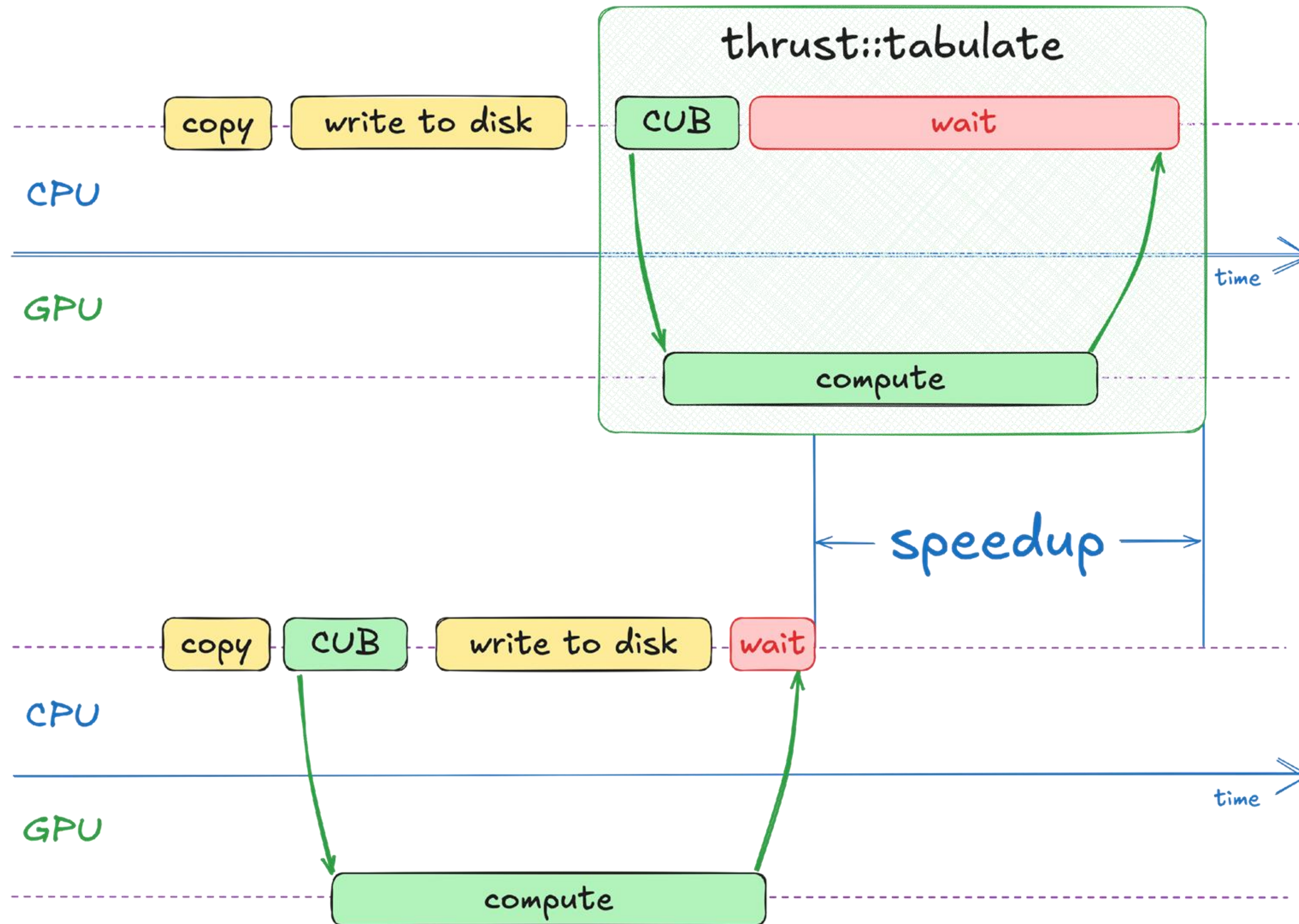
```
auto begin = std::chrono::high_resolution_clock::now();
auto cell_ids = thrust::make_counting_iterator(0);
cub::DeviceTransform::Transform(cell_ids, out.begin(), num_cells, compute);
cudaDeviceSynchronize();
auto end = std::chrono::high_resolution_clock::now();
```

cudaDeviceSynchronize

- CUDA 런타임의 일부입니다.
- GPU에 대한 모든 작업이 완료될 때까지 블록
- CUB 버전도 동기식으로 만듭니다.



동기식 (Synchronous) vs 비동기식 (Asynchronous)



- CUDA 런타임과 함께 비동기 CUB API를 직접 사용하면 CPU가 디스크에 결과를 쓰느라 바쁘게 유지할 수 있습니다.
이렇게 하면 속도가 빨라집니다.

연습: Compute-I/O 중복

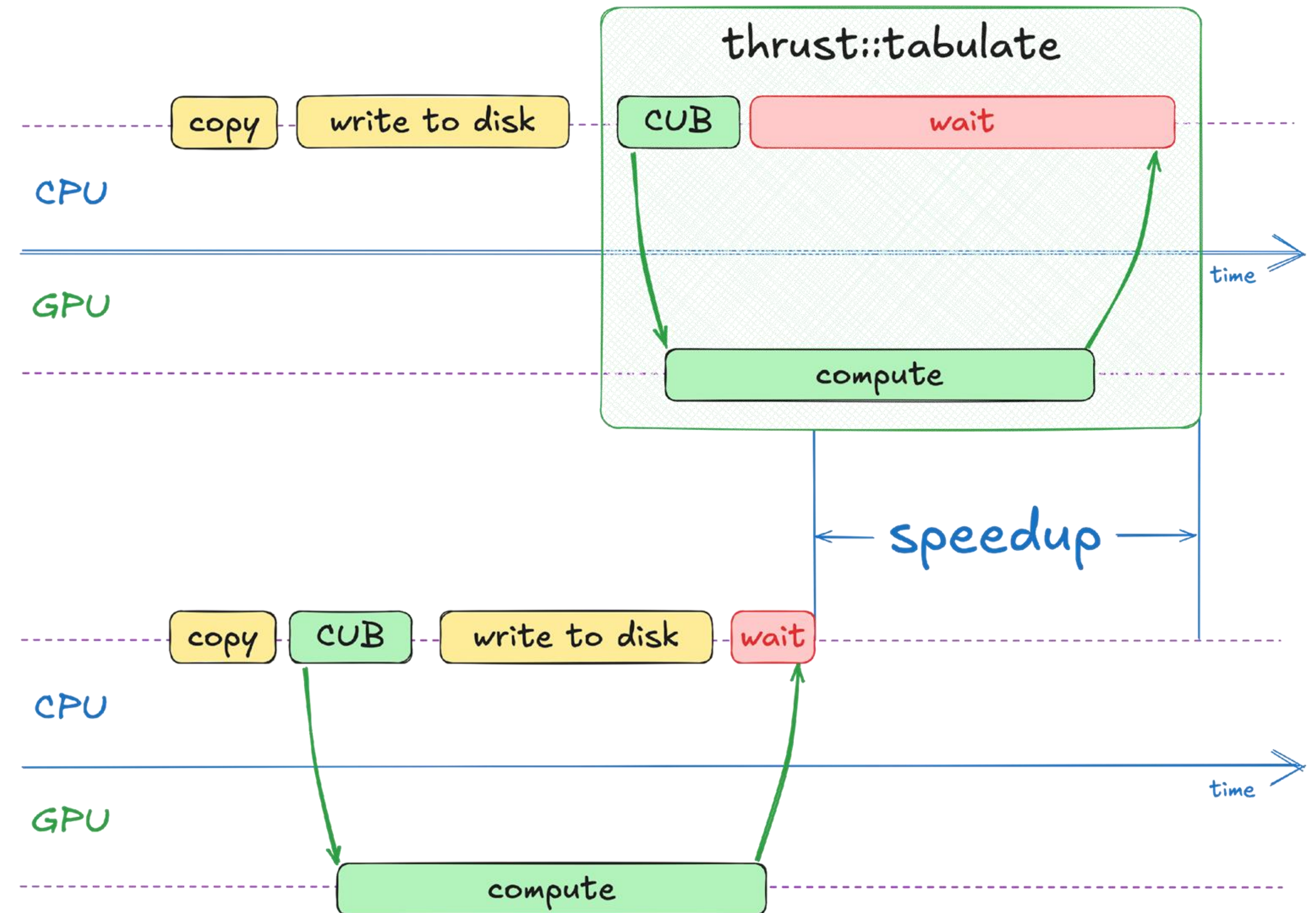
10 분

1. `thrust::tabulate`를 `cub::DeviceTransform::Transform`로 교체합니다.

```
thrust::tabulate(  
    thrust::device, out.begin(), out.end(),  
    compute);
```

```
auto cell_ids =  
    thrust::make_counting_iterator(0);  
cub::DeviceTransform::Transform(  
    cell_ids, out.begin(), num_cells,  
    compute);
```

1. 디스크에 결과를 저장한 후 `cudaDeviceSynchronize`를 추가하여 오른쪽의 구성표와 일치합니다.



연습: Compute-IO 중복

솔루션

- CUB는 변환이 완료될 때까지 기다리지 않고 즉시 제어를 반환합니다.

```
thrust::copy(dprev.begin(), dprev.end(), hprev.begin());

for (int step = 0; step < steps; step++)
{
    simulate(height, width, dprev, dnext); -- ► void simulate(int width,
    dprev.swap(dnext);                        int height,
                                              const thrust::device_vector<float> &in,
                                              thrust::device_vector<float> &out)
{
    // ...
    auto cell_ids = thrust::make_counting_iterator(0);
    cub::DeviceTransform::Transform(
        cell_ids, out.begin(),
        width * height,
        compute);
}

store(write, height, width, hprev);
cudaDeviceSynchronize();
```


연습: Compute-IO 중복

솔루션

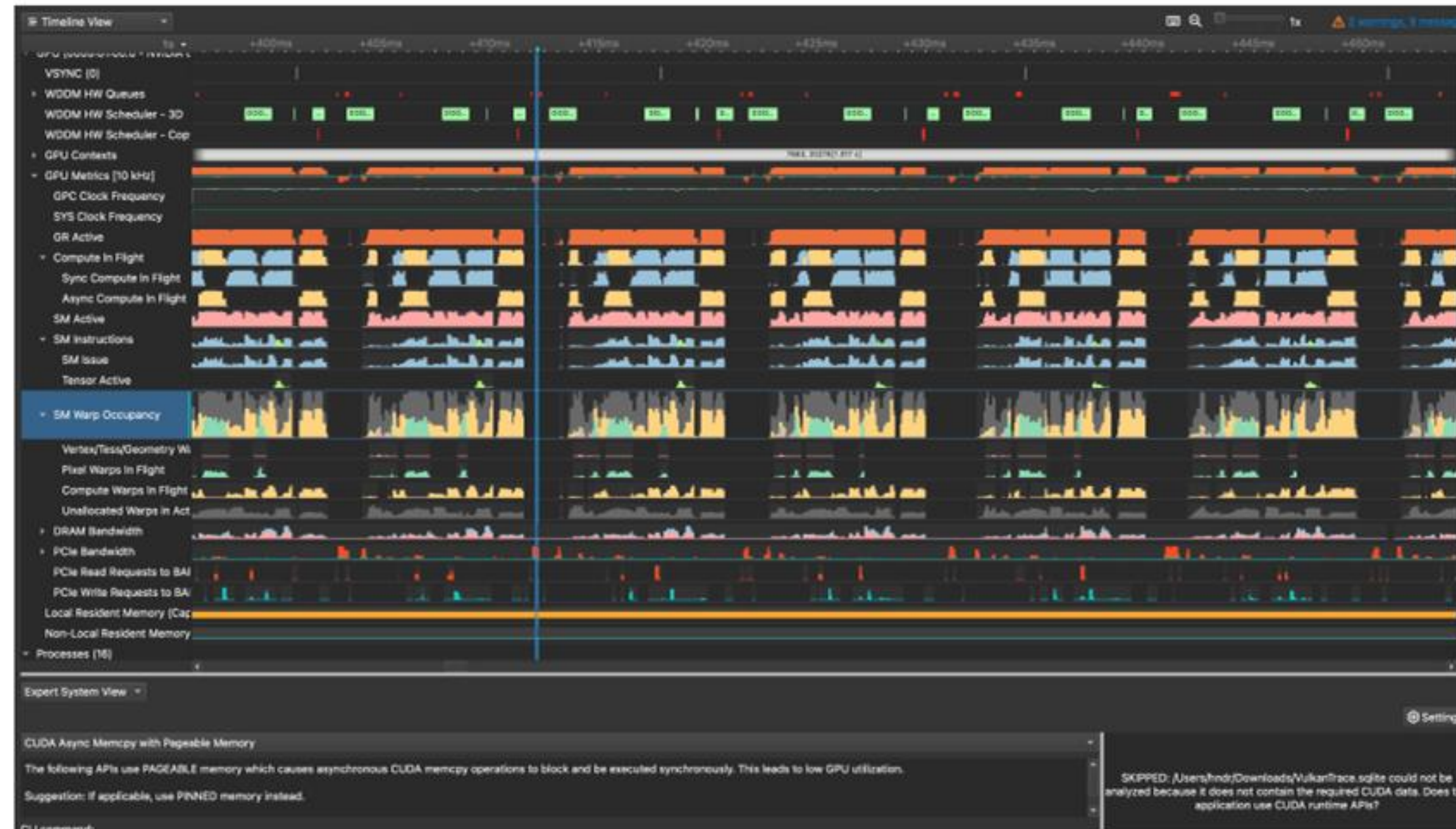
- CUB는 변환이 완료될 때까지 기다리지 않고 즉시 제어를 반환합니다.
- `cudaDeviceSynchronize` 를 명시적으로 추가해야 합니다.

```
thrust::copy(dprev.begin(), dprev.end(), hprev.begin());

for (int step = 0; step < steps; step++)
{
    simulate(height, width, dprev, dnext); -- ► void simulate(int width,
    dprev.swap(dnext);                               int height,
                                                    const thrust::device_vector<float> &in,
                                                    thrust::device_vector<float> &out)
                                                    {
// ...
auto cell_ids = thrust::make_counting_iterator(0);
cub::DeviceTransform::Transform(
    cell_ids, out.begin(),
    width * height,
    compute);
}

store(write, height, width, hprev);
cudaDeviceSynchronize();
```


Nsight Systems



- Nsight Systems는 GPU 및 CPU 활동에 대한 시스템 전체 보기를 제공합니다. 비동기 컴퓨팅 및 메모리 전송을 시각적으로 나타냅니다.
- 이를 통해 최적화 기회를 시각적으로 식별할 수 있습니다.

연습: Nsight Systems를 사용하여 코드 프로파일링

10분

The screenshot shows the NVIDIA Nsight Systems application. On the left is a file explorer for the directory `/ 02.02-Asynchrony /`. It contains a table with the following data:

Name	Modified
Solutions	5 seconds ago
Sources	5 seconds ago
02.02.01-Asynchro...	5 seconds ago
02.02.02-Exercise...	5 seconds ago
02.02.03-Nsight.ip...	now

The main window displays a Jupyter Notebook titled `02.02.03-Nsight.ipynb` in the `Code` view. The code cell [1]: contains the following commands:

```
!nvcc --extended-lambda -o /tmp/a.out Solutions/compute-io-overlap.cu # build executable
!nsys profile -o ../nsight-reports/compute-io-overlap /tmp/a.out # run and profile executable
```

The output of the code cell shows the execution of the CUDA program with Nsight Systems profiling:

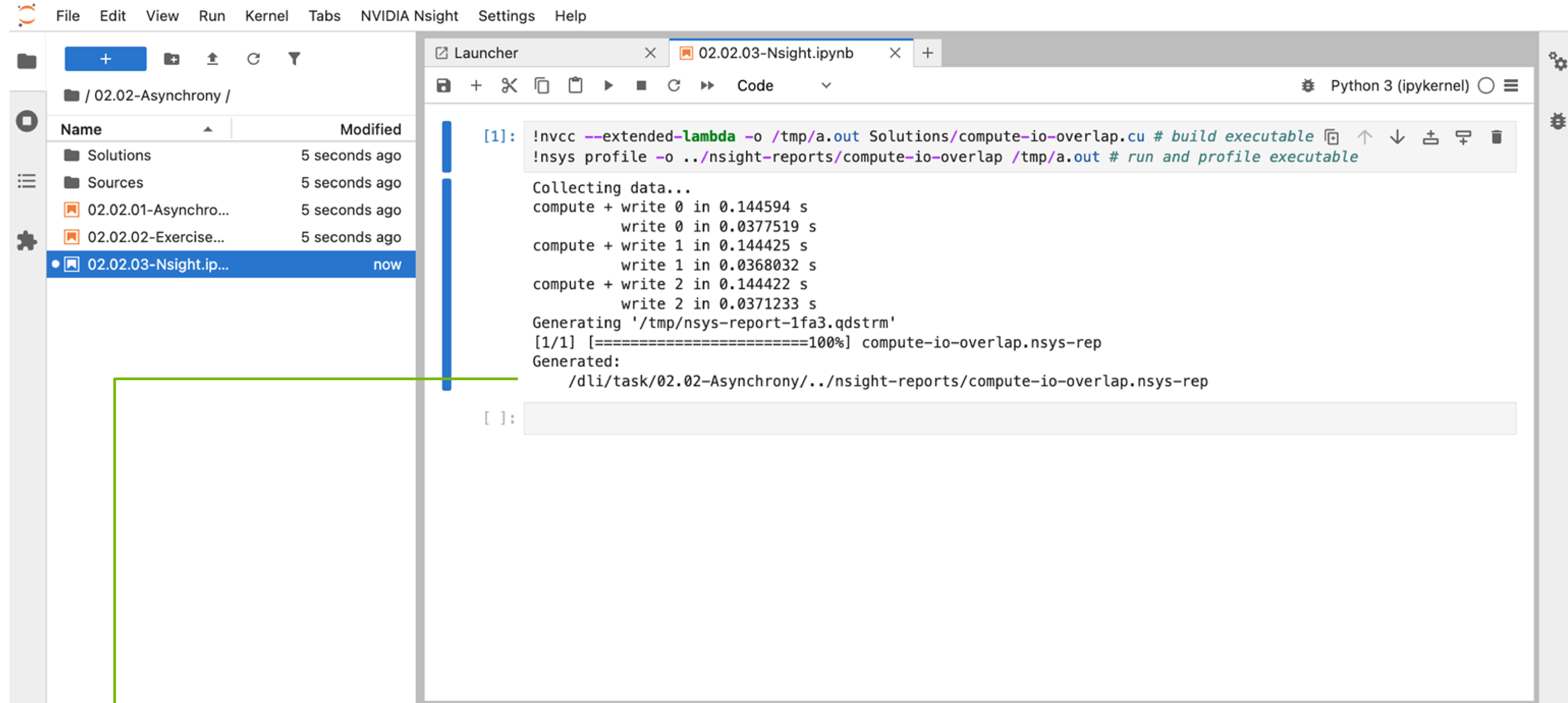
```
Collecting data...
compute + write 0 in 0.144594 s
        write 0 in 0.0377519 s
compute + write 1 in 0.144425 s
        write 1 in 0.0368032 s
compute + write 2 in 0.144422 s
        write 2 in 0.0371233 s
Generating '/tmp/nsys-report-1fa3.qdstrm'
[1/1] [=====100%] compute-io-overlap.nsys-rep
Generated:
        /dli/task/02.02-Asynchrony/./nsight-reports/compute-io-overlap.nsys-rep
```

The code cell []: is currently empty.

02.02-Asynchrony/02.02.03-Exercise-Nsight.ipynb

연습: Nsight Systems를 사용하여 코드 프로파일링

10분



The screenshot shows the NVIDIA Nsight Systems interface. On the left is a file explorer showing a directory structure under '/ 02.02-Asynchrony /'. The main area displays a Jupyter Notebook with a code cell. The code cell contains the following commands:

```
[1]: !nvcc --extended-lambda -o /tmp/a.out Solutions/compute-io-overlap.cu # build executable
!nsys profile -o ../nsight-reports/compute-io-overlap /tmp/a.out # run and profile executable
```

The output of the code cell shows the execution of the program and the generation of a report:

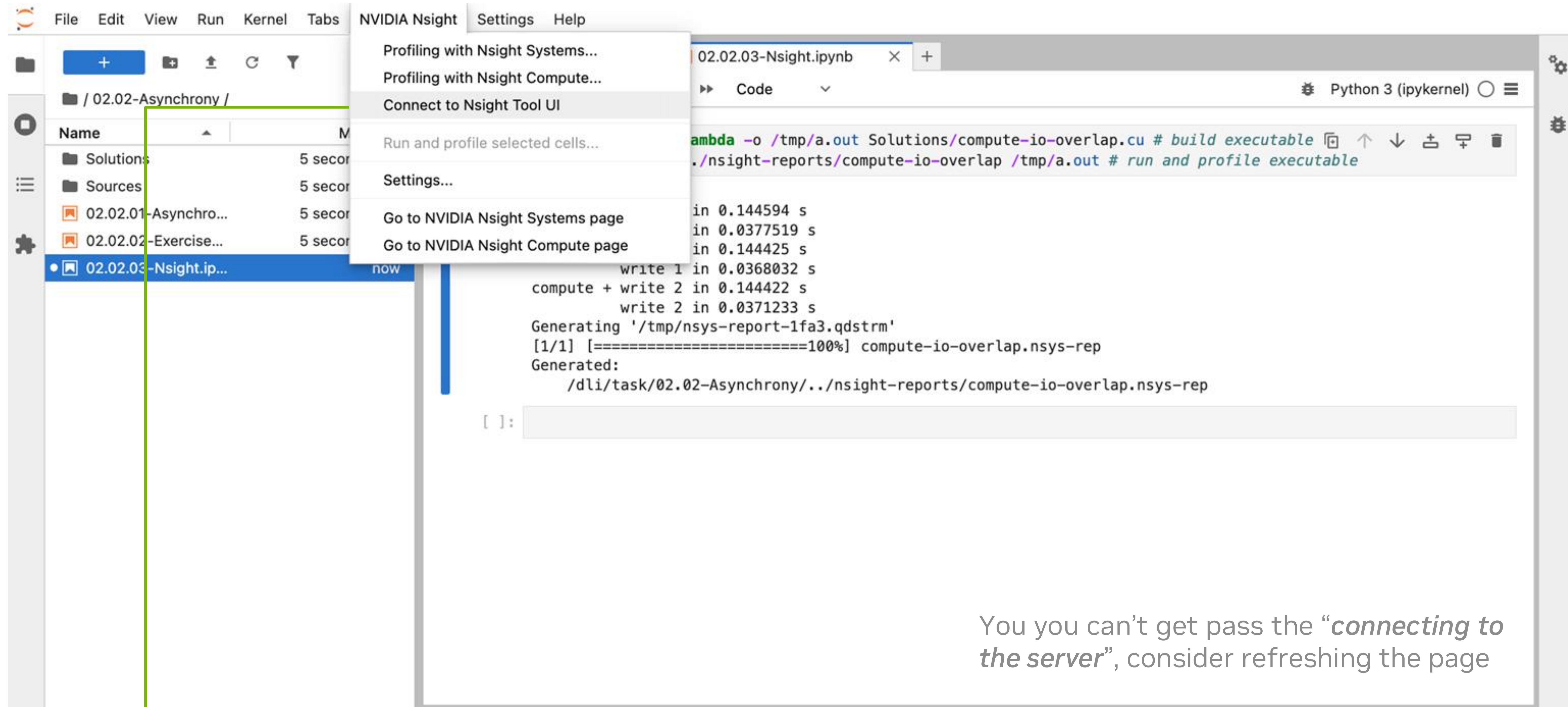
```
Collecting data...
compute + write 0 in 0.144594 s
      write 0 in 0.0377519 s
compute + write 1 in 0.144425 s
      write 1 in 0.0368032 s
compute + write 2 in 0.144422 s
      write 2 in 0.0371233 s
Generating '/tmp/nsys-report-1fa3.qdstrm'
[1/1] [=====100%] compute-io-overlap.nsys-rep
Generated:
      /dli/task/02.02-Asynchrony/./nsight-reports/compute-io-overlap.nsys-rep
```

A green line connects the text 'Profile your app with nsys profile and store the report' to the code cell, and another green line connects the text 'file' to the file explorer.

Profile your app with nsys profile and store the report

file

연습: Nsight Systems를 사용하여 코드 프로파일링

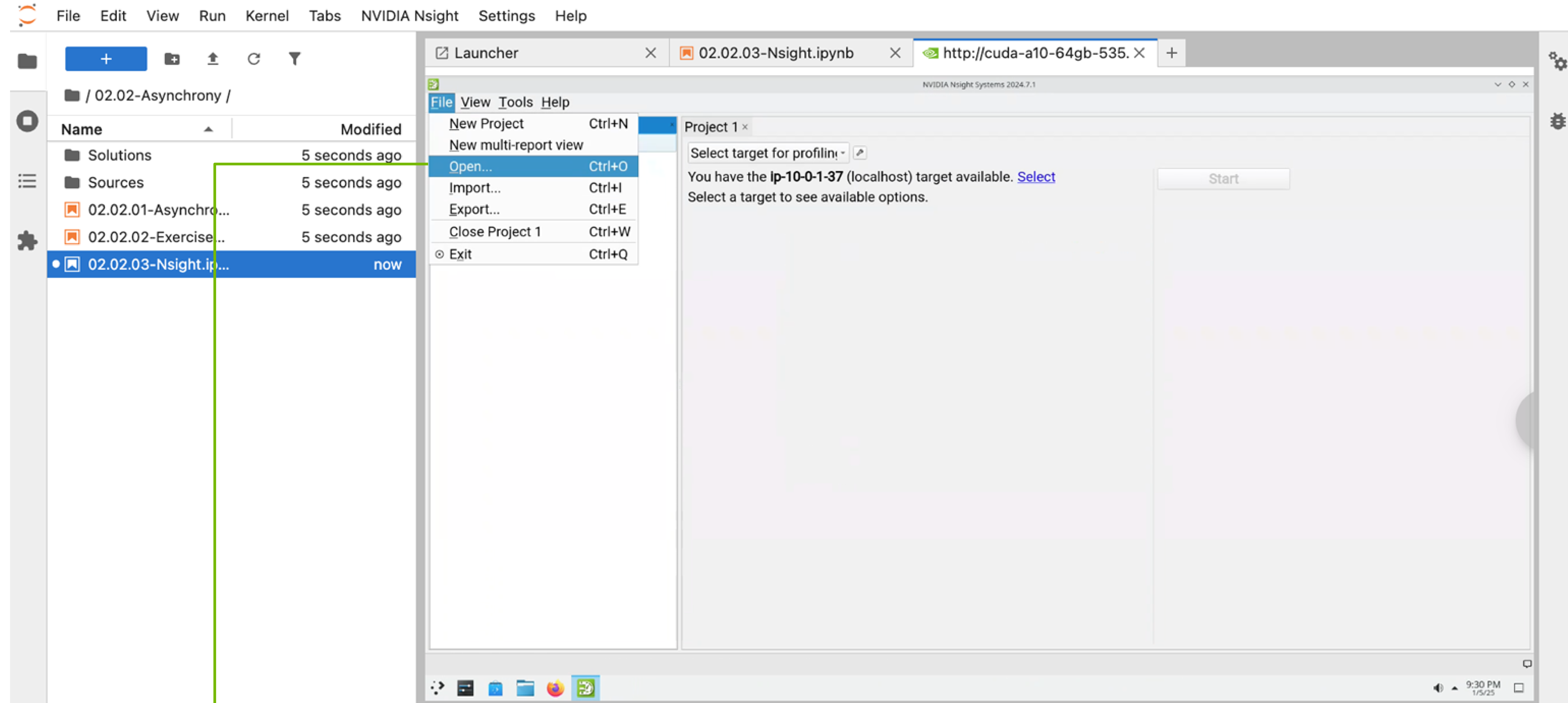


The screenshot displays the NVIDIA Nsight Systems interface. On the left, a file explorer shows a directory structure with files like '02.02.01-Asynchro...' and '02.02.02-Exercise...'. The main area shows a Jupyter Notebook with a code cell containing CUDA kernel code. The code includes a build step using 'nvcc' and a run step using 'nvidia-smi' and 'nvidia-nsight-systems'. The output of the code cell shows timing information and a generated report path: '/dli/task/02.02-Asynchrony/./nsight-reports/compute-io-overlap.nsys-rep'. The 'NVIDIA Nsight' menu is open, showing options like 'Profiling with Nsight Systems...', 'Profiling with Nsight Compute...', 'Connect to Nsight Tool UI', 'Run and profile selected cells...', 'Settings...', 'Go to NVIDIA Nsight Systems page', and 'Go to NVIDIA Nsight Compute page'. The 'Connect to Nsight Tool UI' option is highlighted.

You you can't get pass the “*connecting to the server*”, consider refreshing the page

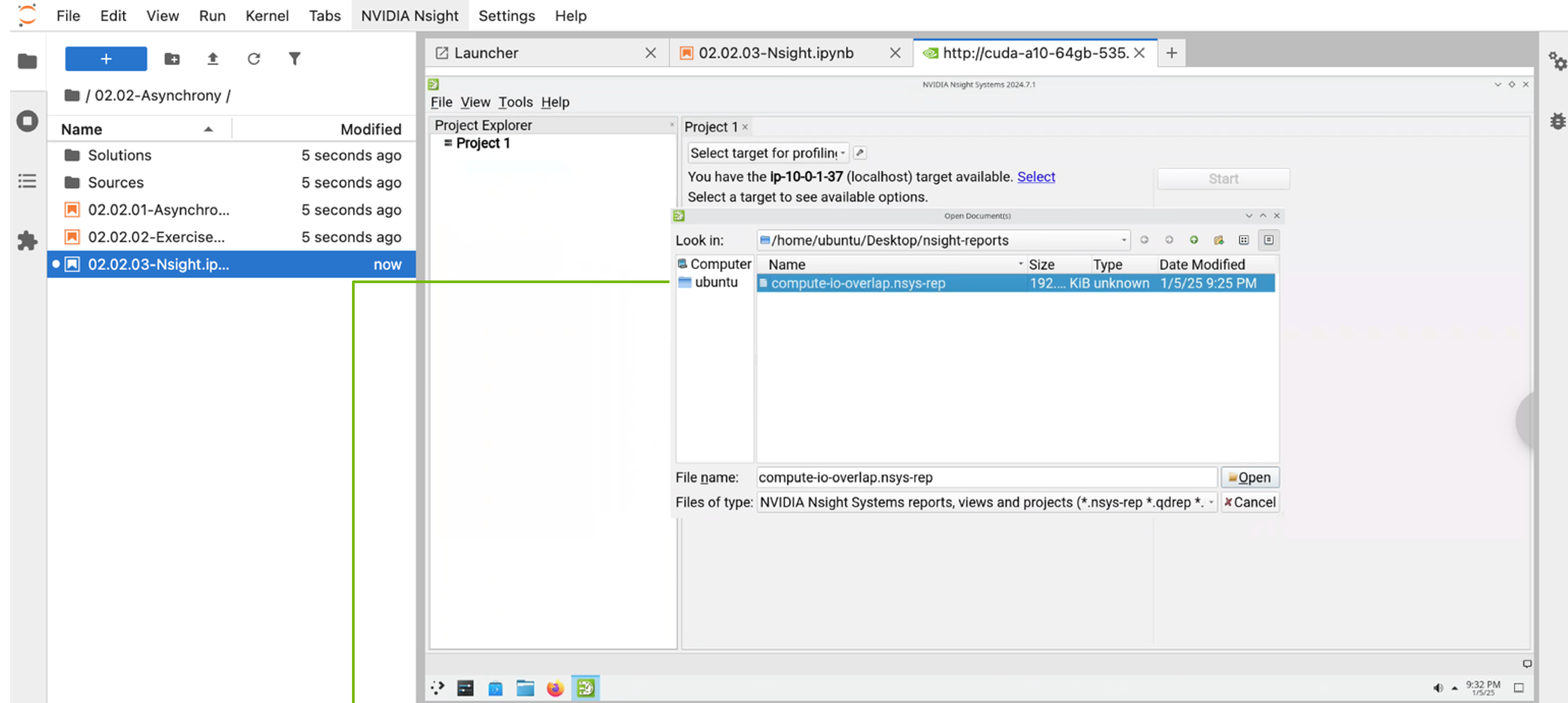
“Connect to Nsight Tool UI” 을 클릭합니다. (“NVIDIA Nsight” 탭)

연습: Nsight Systems를 사용하여 코드 프로파일링



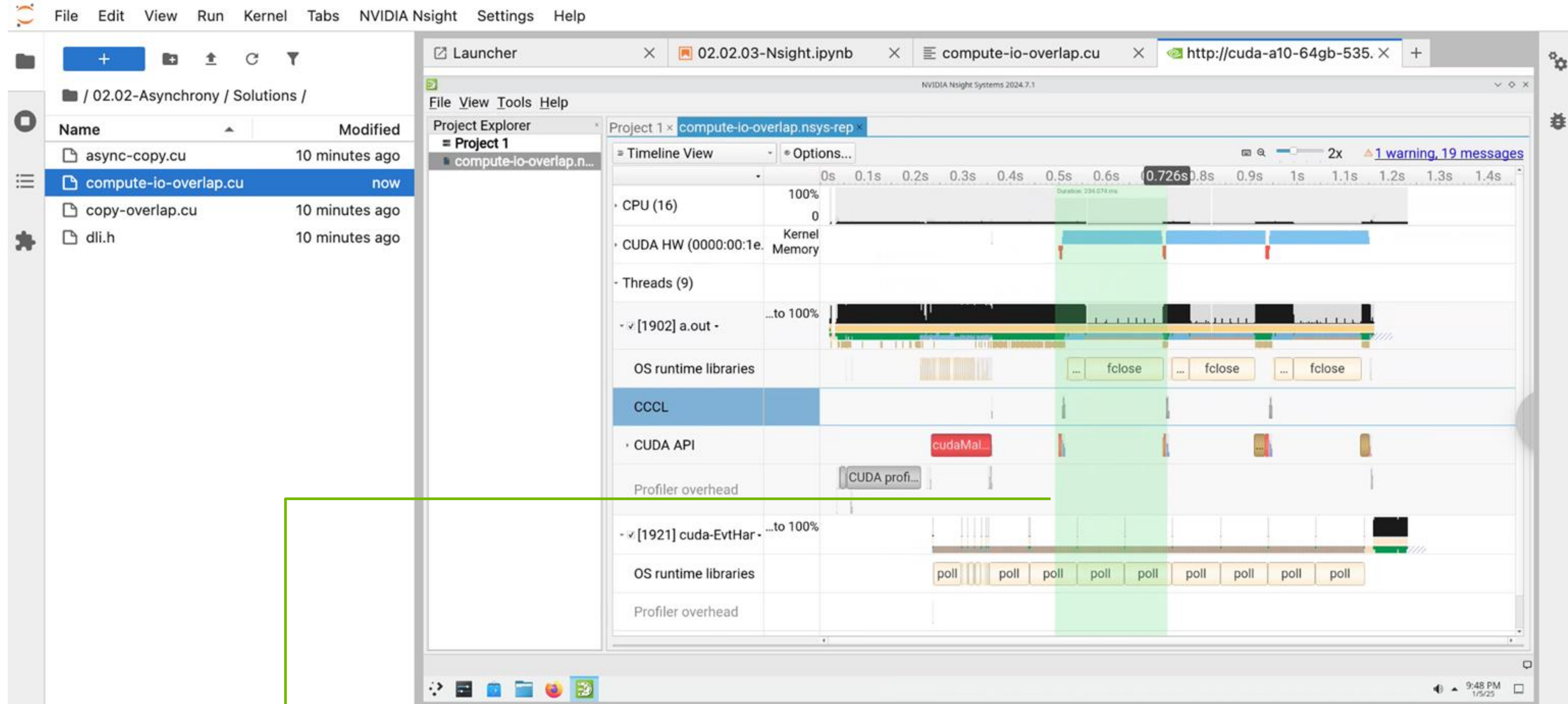
Nsight Systems를 엽니다: File - Open...

연습: Nsight Systems를 사용하여 코드 프로파일링



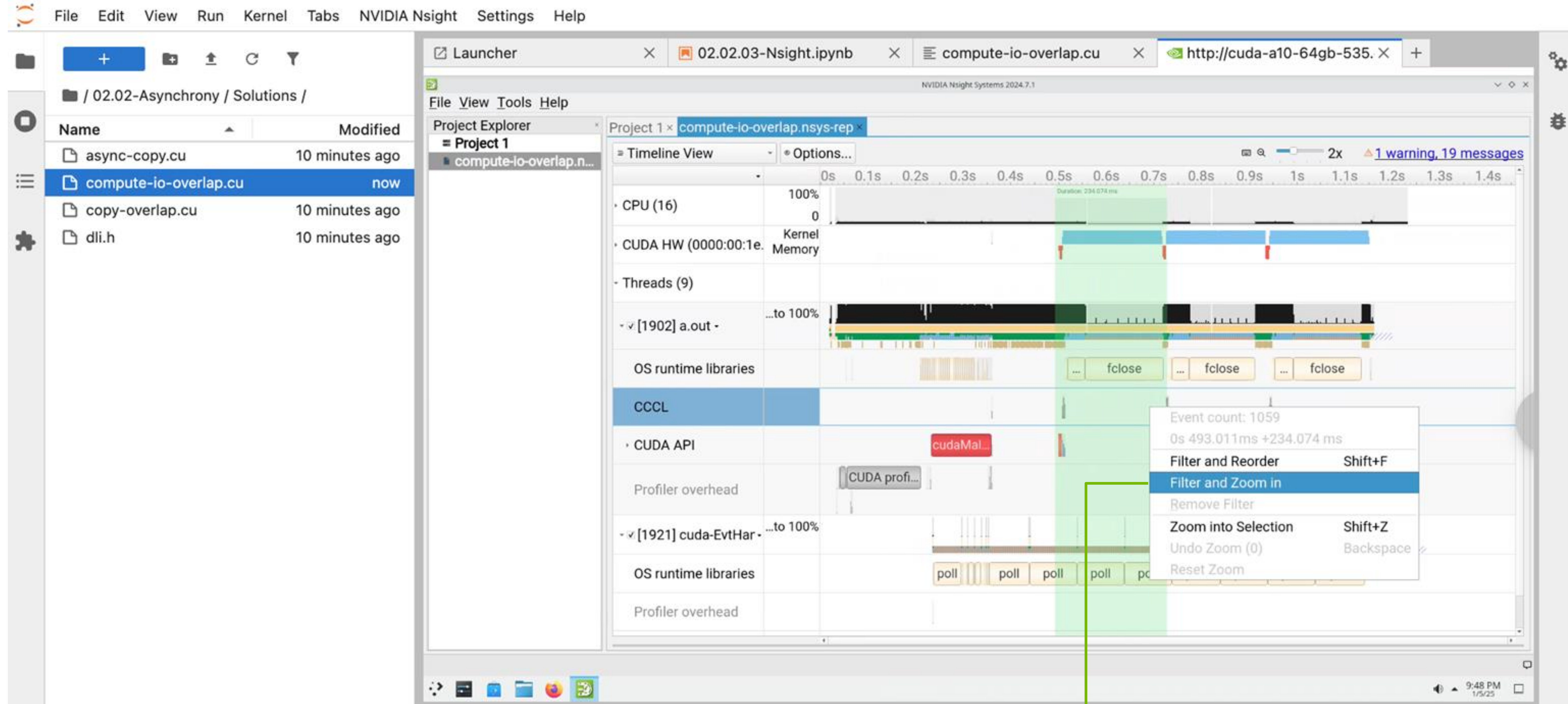
방금 저장한 보고서로 이동합니다.

연습: Nsight Systems를 사용하여 코드 프로파일링



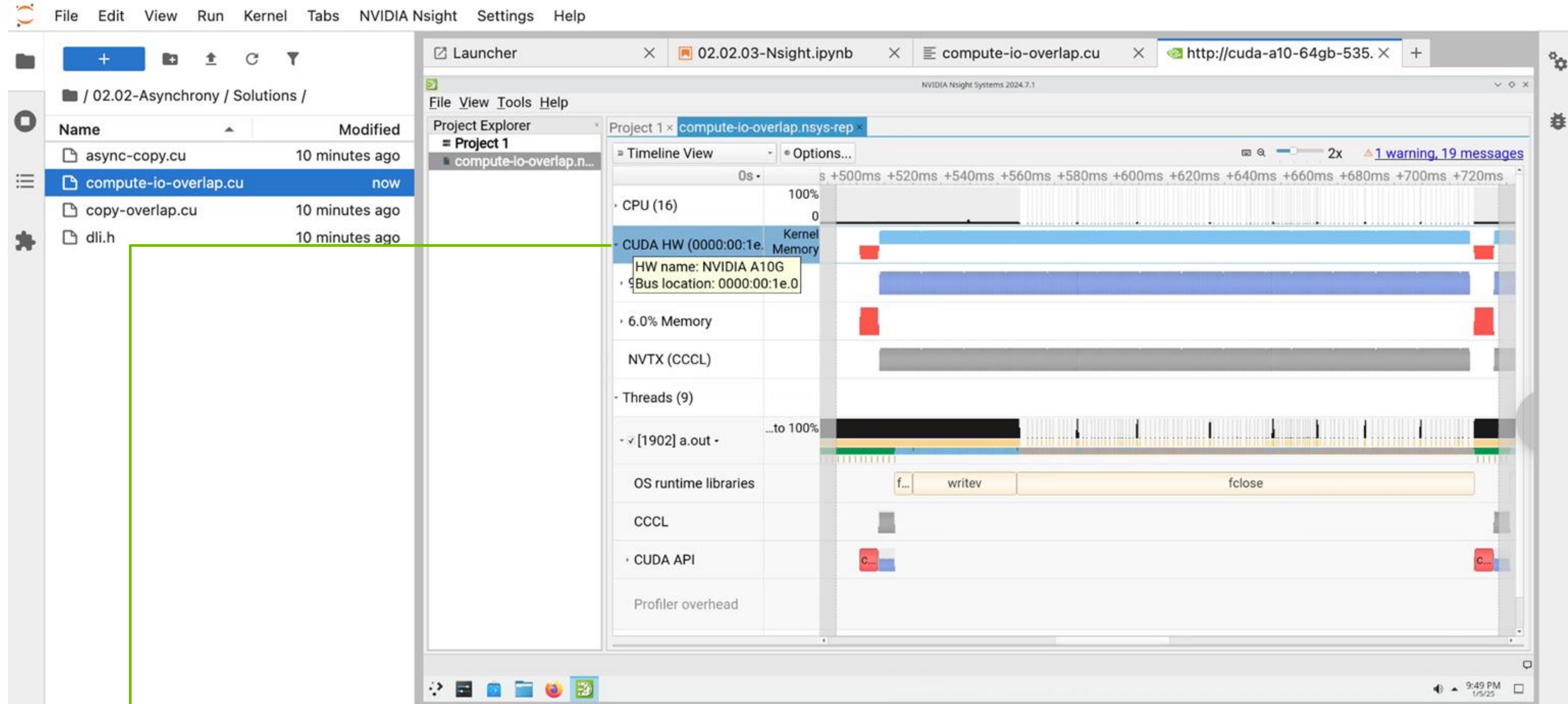
클릭하고 드래그하여 시간 영역을 선택합니다.

연습: Nsight Systems를 사용하여 코드 프로파일링



“Filter and Zoom in” 클릭

연습: Nsight Systems를 사용하여 코드 프로파일링



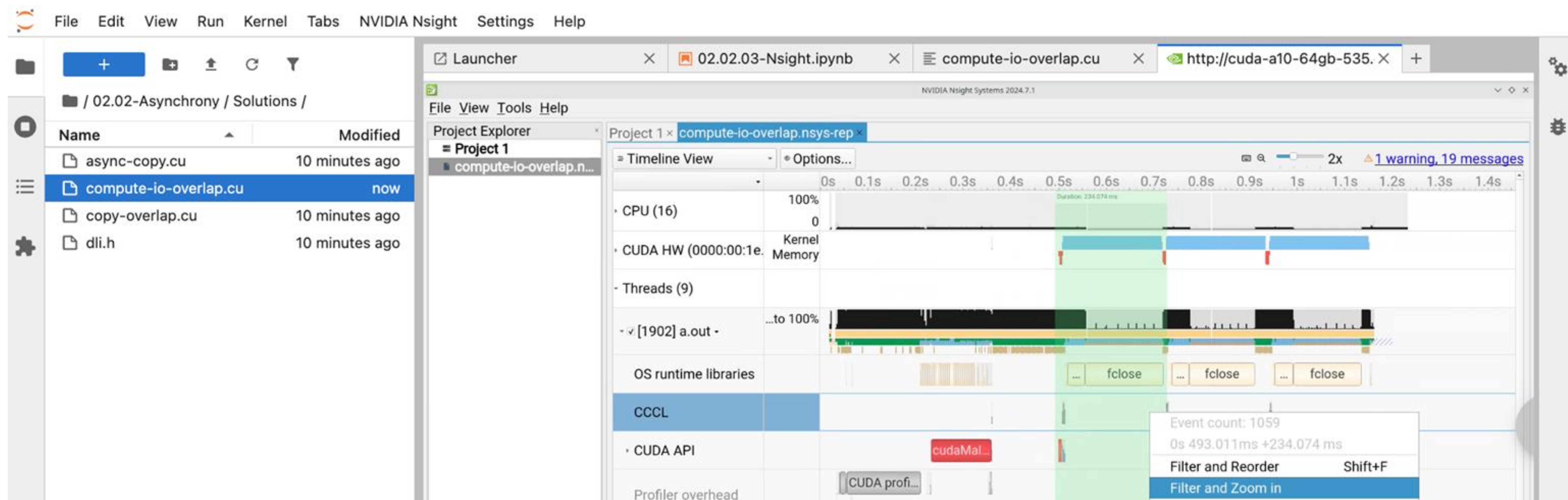
"CUDA HW"를 펼치면 GPU가 무엇을 하고 있는지에 대한 자세한 내용을 볼 수 있습니다.

연습 문제: Nsight Systems로 코드 프로파일링 (Profiling)

10 분

Desktop/nsight-reports/compute-io-overlap.nsys-rep의 타임라인 (timeline)을 살펴보세요:

- GPU 연산 (compute)이 시작되는 시점
- CPU가 GPU를 기다리는 시점
- CPU가 디스크에 데이터를 기록하는 시점
- 데이터가 전송되는 시점

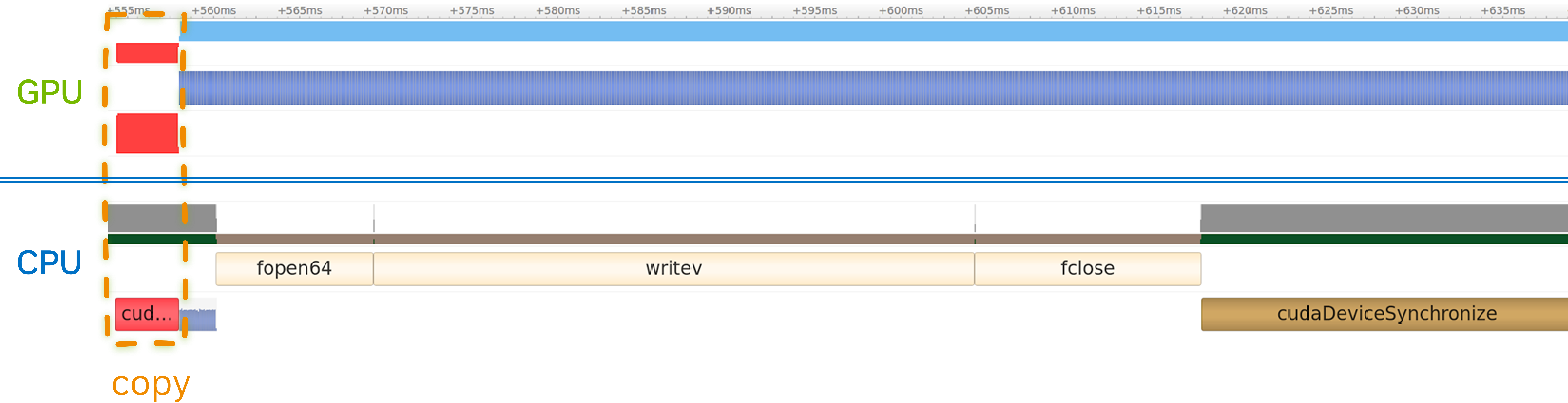


02.02-Asynchrony/02.02.03-Exercise-Nsight.ipynb

연습: Nsight를 사용하여 코드 프로파일링

솔루션

compute

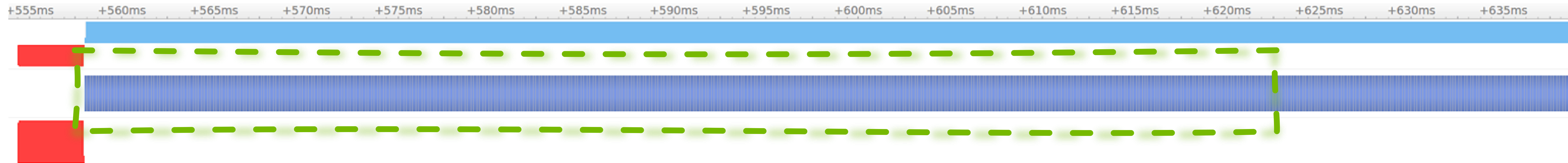


연습: Nsight를 사용하여 코드 프로파일링

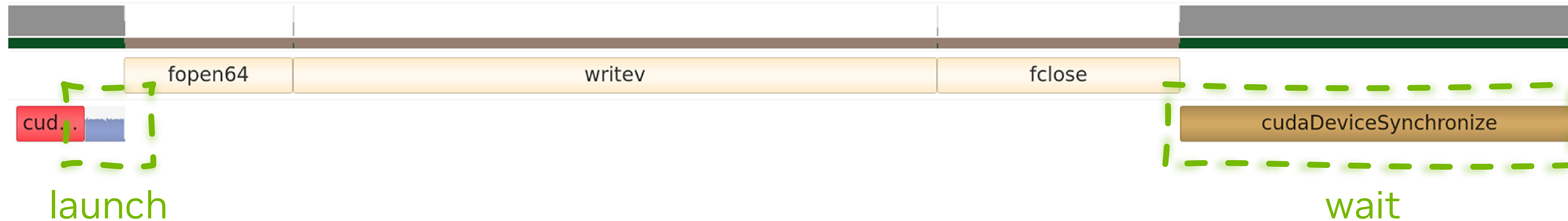
솔루션

compute

GPU

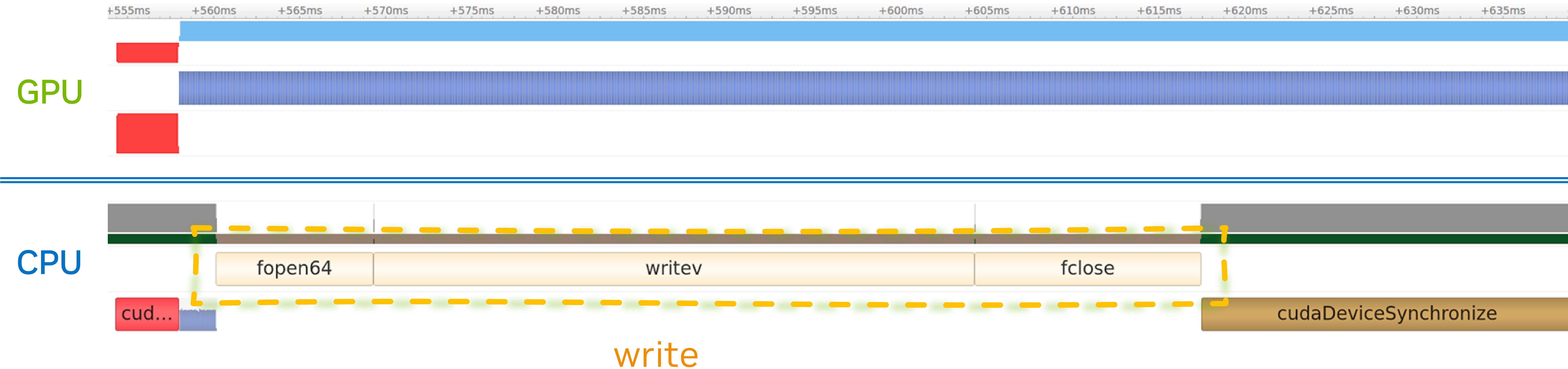


CPU



- 알고리즘은 이제 비동기식입니다.
이제 GPU와의 동기화가 명시적입니다.

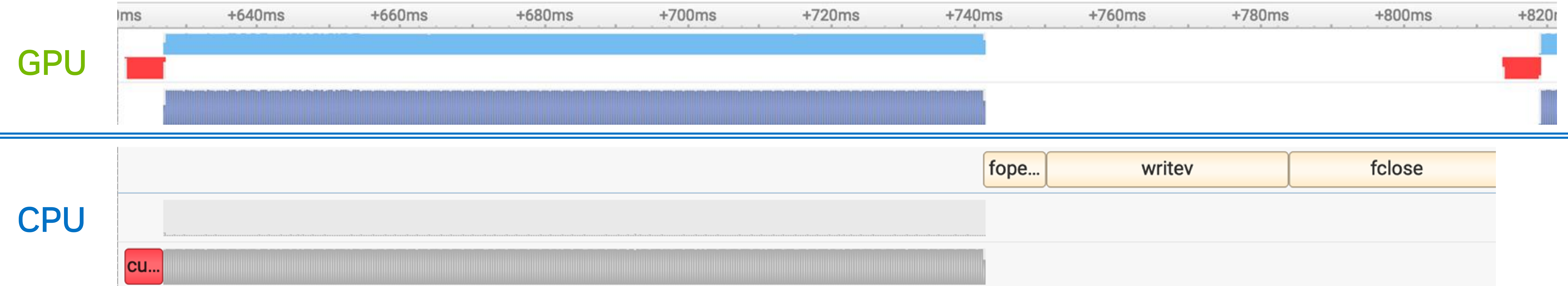
연습: Nsight를 사용하여 코드 프로파일링



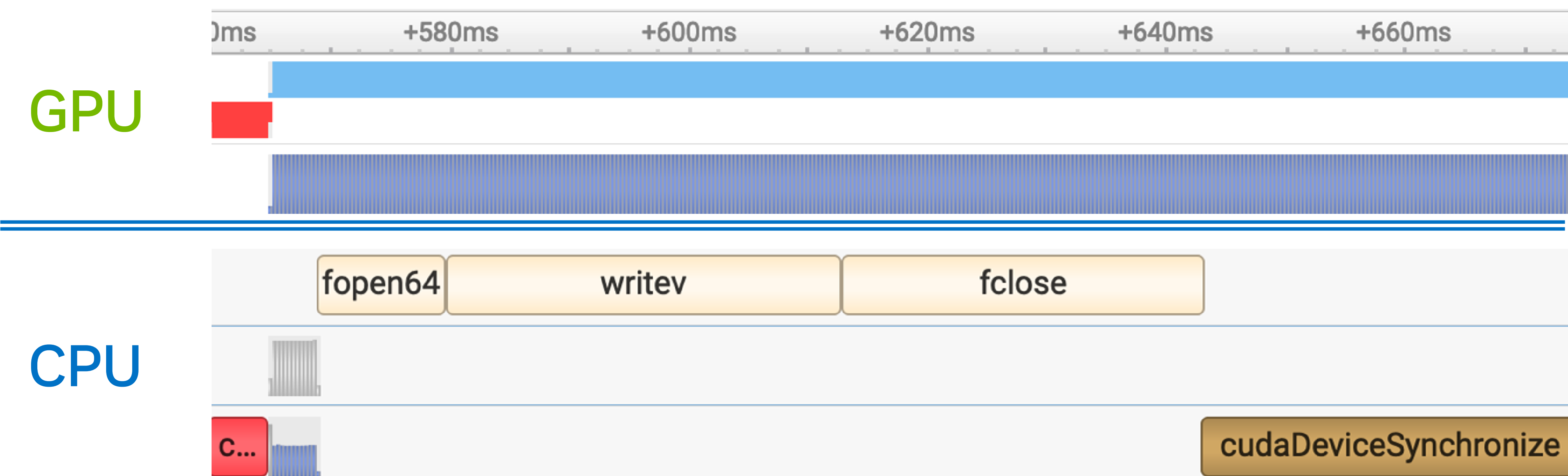
- 알고리즘은 이제 비동기식입니다.
- 이제 GPU와의 동기화가 명시적입니다.
- GPU 연산은 이제 CPU의 쓰기와 겹칩니다.

Synchronous vs Asynchronous

Synchronous



Asynchronous



- 시뮬레이션에는 IO를 완전히 겹칠 수 있는 충분한 컴퓨팅이 있습니다.
- 이렇게하면 컴퓨팅 IO가 겹치는 버전이 거의 두 배 더 빠릅니다.

NVIDIA Tools Extension (NVTX)

<https://github.com/NVIDIA/NVTX>

- 복잡한 애플리케이션에 대한 Nsight 보고서를 검사하는 것은 부담스러울 수 있습니다.
- NVTX를 사용하면 코드에서 Nsight Systems의 사용자 지정 범위를 생성할 수 있습니다.

```
int main()  
{  
    for (int write_step = 0; write_step < write_steps; write_step++)  
    {  
        nvtx3::scoped_range r{std::string("write step ") + std::to_string(write_step)};  
        ...  
    }  
}
```

fopen64

writew

fclose

write step 0 [76.706 ms]

연습: NVTX 사용

8분

- `nvtx3::scoped_range` 를 사용하여 복사, 계산, 쓰기 및 대기 단계에 주석을 추가합니다.
- Nsight Systems를 사용하여 애플리케이션을 프로파일링하고 지정된 범위를 찾습니다.

```
int main()
{
    for (int write_step = 0; write_step < write_steps; write_step++)
    {
        nvtx3::scoped_range r{std::string("write step ") + std::to_string(write_step)};
        ...
    }
}
```

fopen64

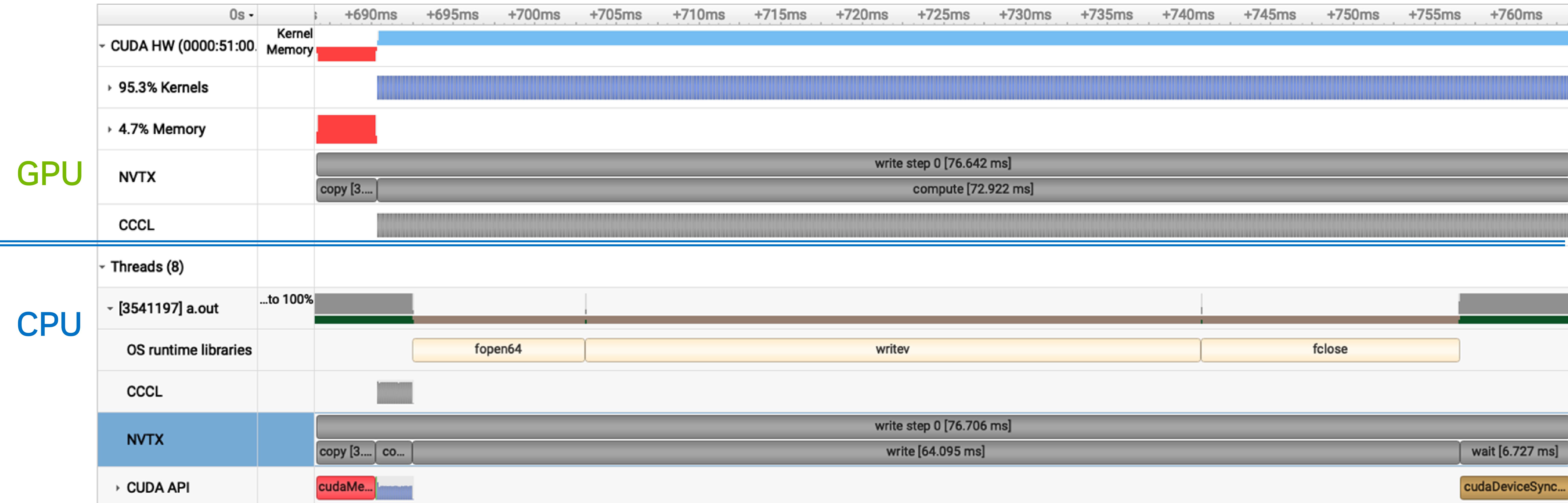
writew

fclose

write step 0 [76.706 ms]

연습: NVTX 사용

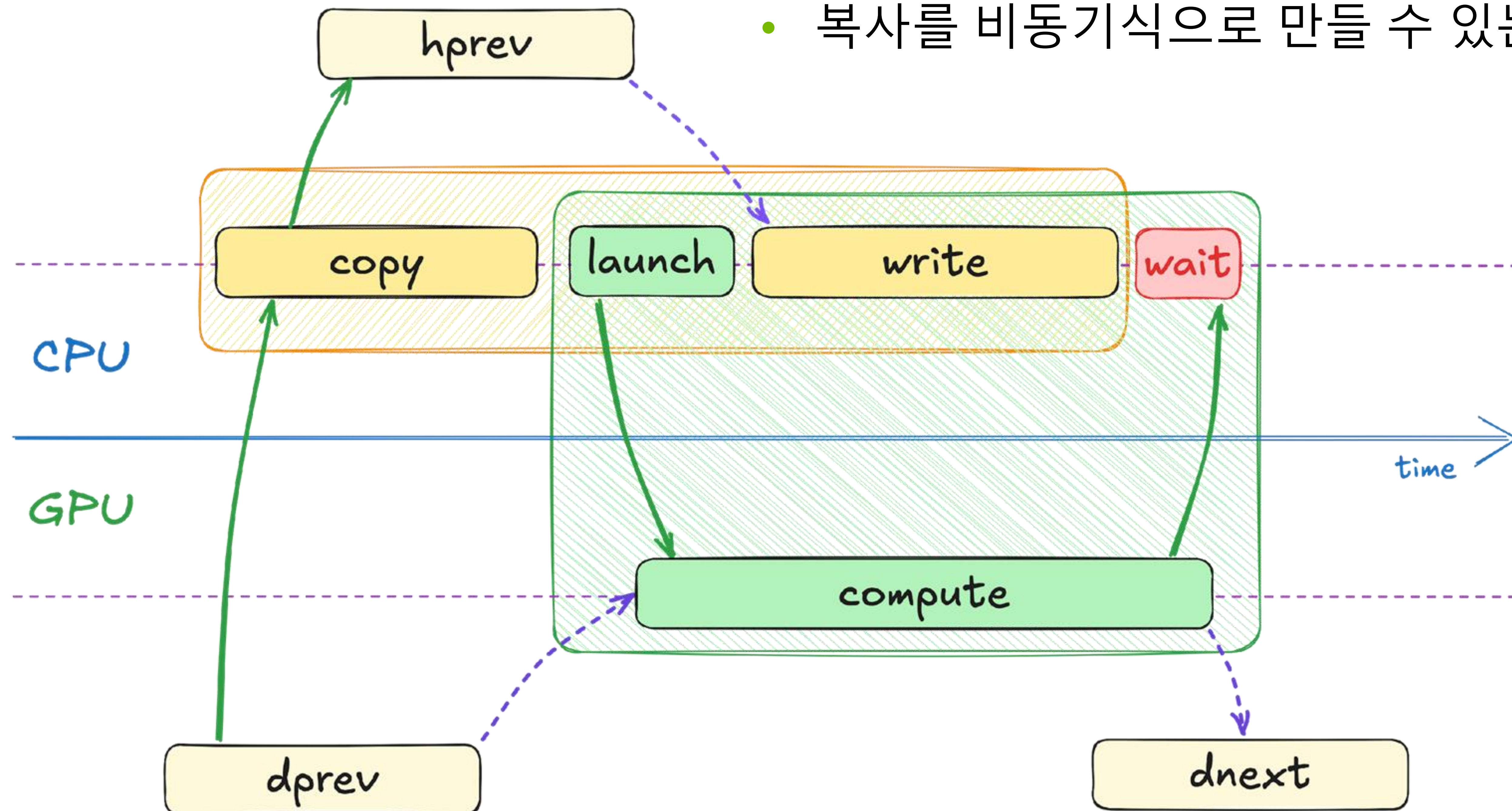
솔루션



- 이제 모든 단계를 훨씬 쉽게 찾을 수 있습니다.

CUDA 스트림

- 컴퓨팅이 복사가 완료될 때까지 불필요하게 기다리는 방법을 관찰합니다.
- 복사를 비동기식으로 만들 수 있는 `cudaMemcpyAsync`가 있습니다.



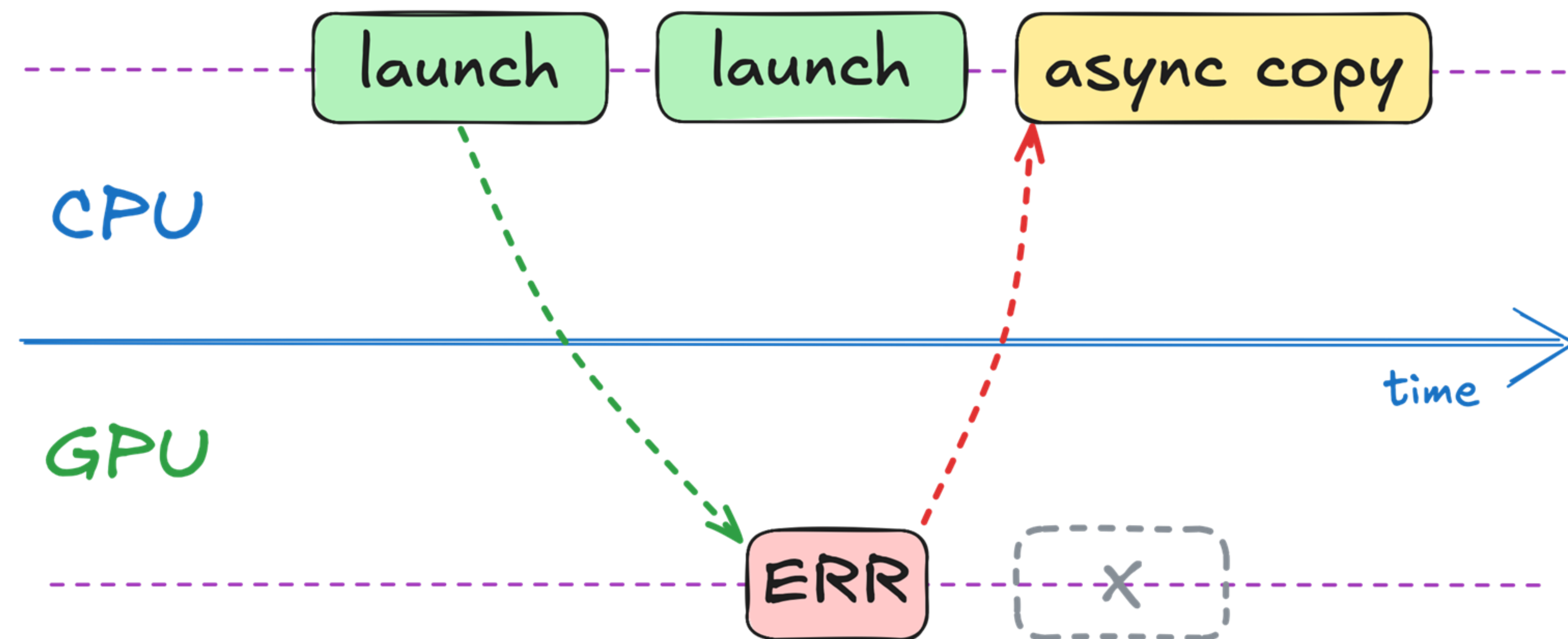
CUDA 스트림

- `cudaMemcpyAsync`는 `thrust::copy` 뒤에 숨은 마법입니다
- 주의:
 - 형식 저장이 아님
 - 요소가 아닌 바이트로 작동합니다.
 - 명시적으로 지정된 복사 방향
 - 비동기 오류는 놓치기 쉽습니다.

```
cudaError_t cudaMemcpyAsync(  
    void*          dst, —————→ Destination memory address  
    const void*    src, —————→ Source memory address  
    size_t         count, —————→ Size in bytes to copy  
    cudaMemcpyKind kind —————→ cudaMemcpyHostToHost  
                                cudaMemcpyHostToDevice  
                                cudaMemcpyDeviceToHost  
                                cudaMemcpyDeviceToDevice  
);
```


CUDA 오류

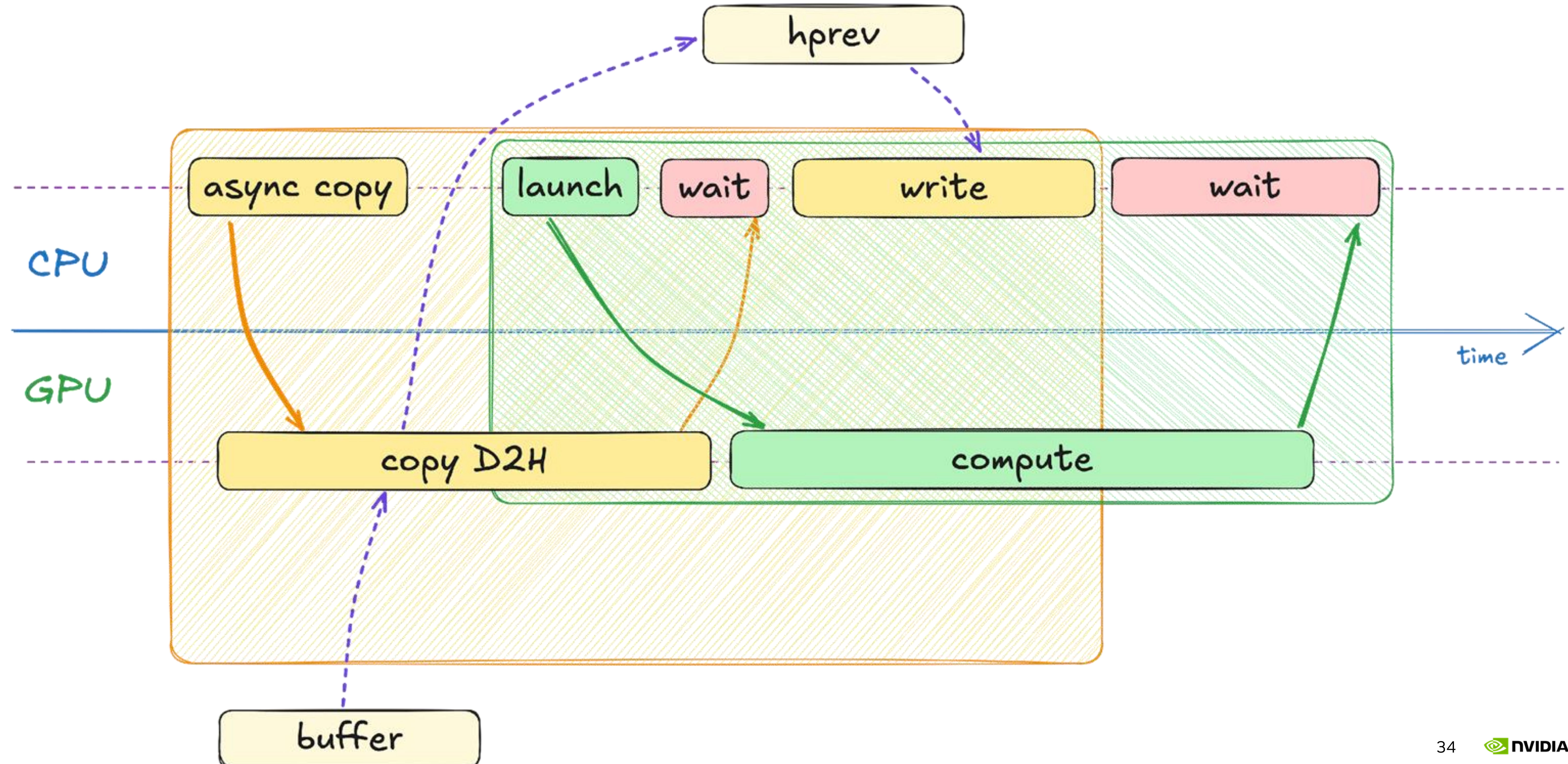
- `cudaMemcpyAsync` 는 어떤 종류의 오류를 반환합니까?
- 이 오류는 이전의 비동기 작업으로 인해 발생할 수 있습니다



```
cudaError_t cudaMemcpyAsync(  
    void*          dst,  
    const void*    src,  
    size_t         count,  
    cudaMemcpyKind kind  
);
```

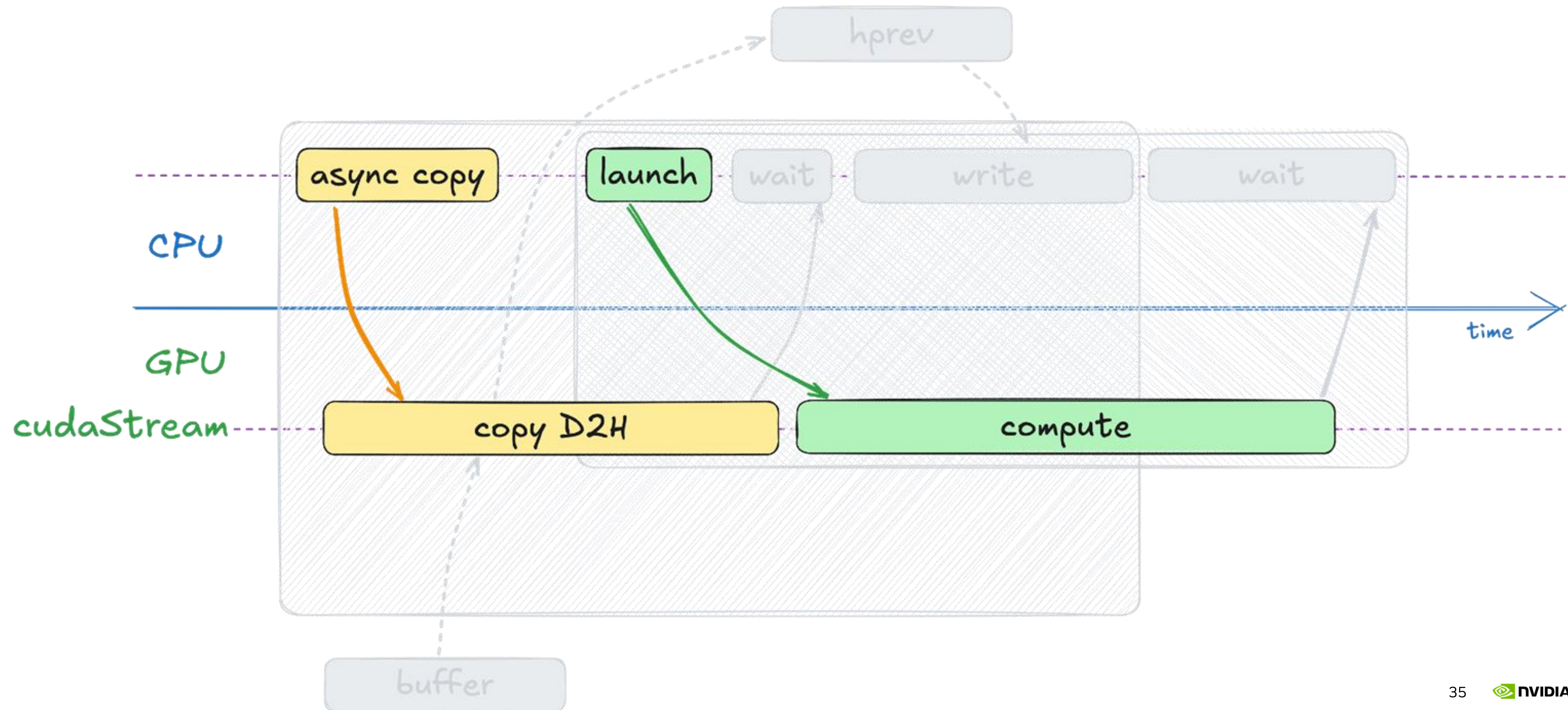

CUDA 스트림

- 이 설정에서 `cudaMemcpyAsync` 는 속도 향상을 제공하지 않습니다.
- 제출된 모든 연산은 GPU에서 진행됩니다.



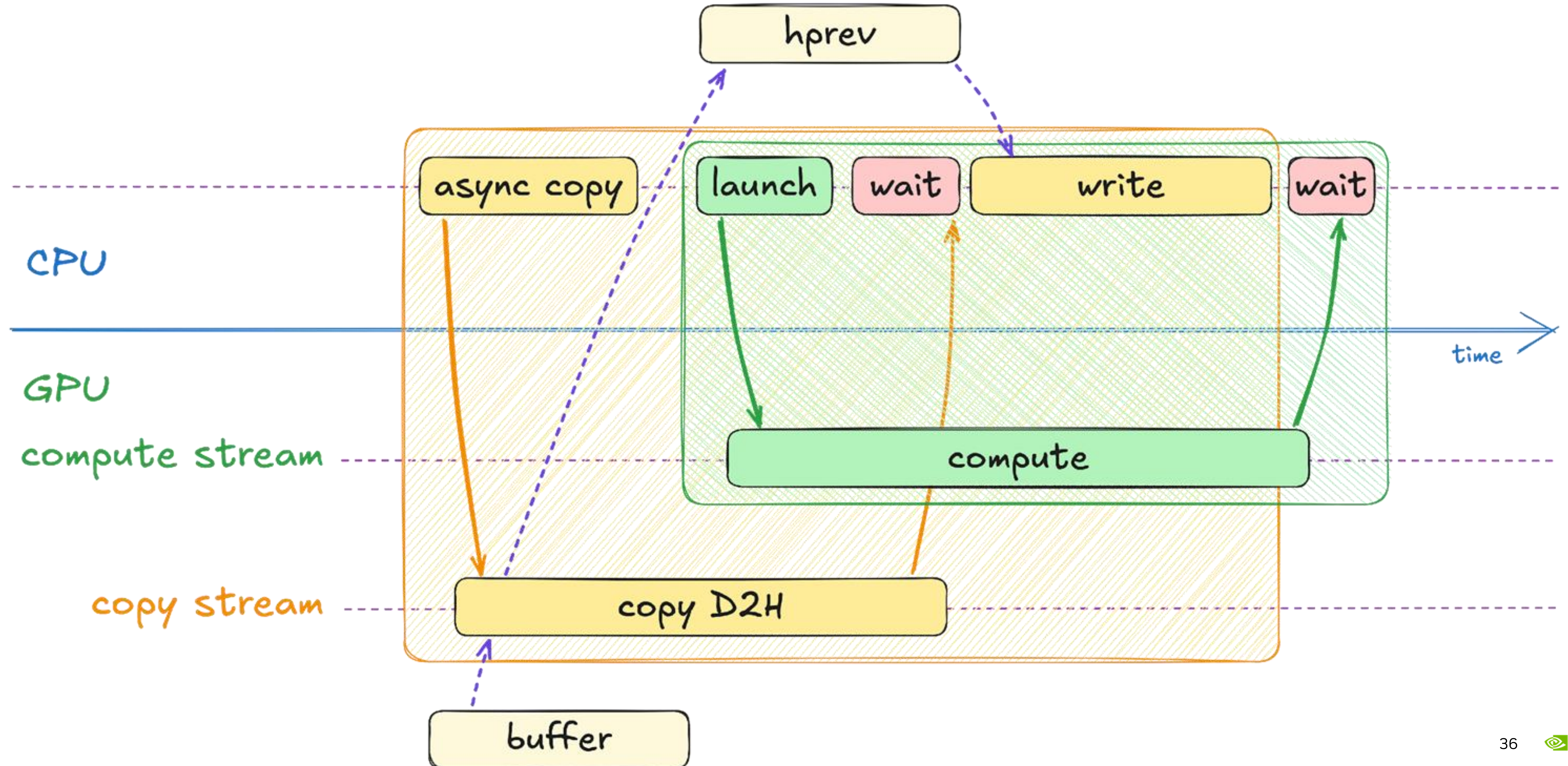
CUDA 스트림

- GPU에 대한 이러한 작업 순서를 **cudaStream**이라고 합니다.
- 스트림을 지정하지 않으면 기본 스트림이 사용됩니다.



CUDA 스트림

- 하나 이상의 스트림을 가질 수 있습니다.
- 서로 다른 스트림에 대한 작업을 동시에 실행할 수 있습니다.



스트림에 대한 작업

```
cudaStream_t copy_stream, compute_stream;
```

I Construction

```
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

II Destruction

```
cudaStreamDestroy(compute_stream);  
cudaStreamDestroy(copy_stream);
```

III Synchronization

```
cudaStreamSynchronize(compute_stream);  
cudaStreamSynchronize(copy_stream);
```

- 스트림의 모든 이전 명령이 완료될 때까지 대기합니다.
- 전체 GPU를 동기화하는 것에 비해 더 가볍습니다.

CUDA 스트림 사용

CUDA Runtime

```
cudaError_t  
cudaMemcpyAsync(  
    void*          dst,  
    const void*    src,  
    size_t         count,  
    cudaMemcpyKind kind,  
    cudaStream_t stream = 0  
);
```

CUB

```
cudaError_t  
cub::DeviceTransform::Transform(  
    IteratorIn  input,  
    IteratorOut output,  
    int         num_items,  
    TransformOp op,  
    cudaStream_t stream = 0  
);
```

cuBLAS

```
cublasStatus_t  
cublasLtMatmul(  
    cublasLtHandle_t    lightHandle,  
    cublasLtMatmulDesc_t computeDesc,  
    const void          *alpha,  
    const void          *A,  
    ...  
    cudaStream_t        stream  
);
```

- 대부분의 비동기 CUDA 라이브러리는 `cudaStream_t` 활용
- 아이디어는 API를 다음과 겹치게 할 수 있다는 것입니다.
 - 메모리 전송,
 - 호스트 사이드 컴퓨팅 또는 IO,
 - 또는 다른 디바이스 사이드 컴퓨팅!

CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;  
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

복사 및 컴퓨트 스트림 만들기

```
// ...  
  
cudaMemcpyAsync(hprev_ptr,  
                dprev_ptr,  
                num_cells * sizeof(float),  
                cudaMemcpyDeviceToHost,  
                copy_stream);  
  
for (int step = 0; step < steps; step++)  
{  
    simulate(width, height, dprev, dnext, compute_stream);  
    dprev.swap(dnext);  
}  
  
cudaStreamSynchronize(copy_stream);  
  
store(write_step, height, width, hprev);  
cudaStreamSynchronize(compute_stream);
```


CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;  
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

```
// ...
```

```
cudaMemcpyAsync(hprev_ptr,  
                dprev_ptr,  
                num_cells * sizeof(float),  
                cudaMemcpyDeviceToHost,  
                copy_stream);
```

복사 스트림에서 비동기 cudaMemcpyAsync 를 사
용하여 호스트에 디바이스 벡터 복사

```
for (int step = 0; step < steps; step++)  
{  
    simulate(width, height, dprev, dnext, compute_stream);  
    dprev.swap(dnext);  
}
```

```
cudaStreamSynchronize(copy_stream);
```

```
store(write_step, height, width, hprev);  
cudaStreamSynchronize(compute_stream);
```


CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

// ...

cudaMemcpyAsync(hprev_ptr,
                dprev_ptr,
                num_cells * sizeof(float),
                cudaMemcpyDeviceToHost,
                copy_stream);

for (int step = 0; step < steps; step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);

store(write_step, height, width, hprev);
cudaStreamSynchronize(compute_stream);
```

컴퓨트 스트림에서 CUB 변환 시작

CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;  
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

```
// ...
```

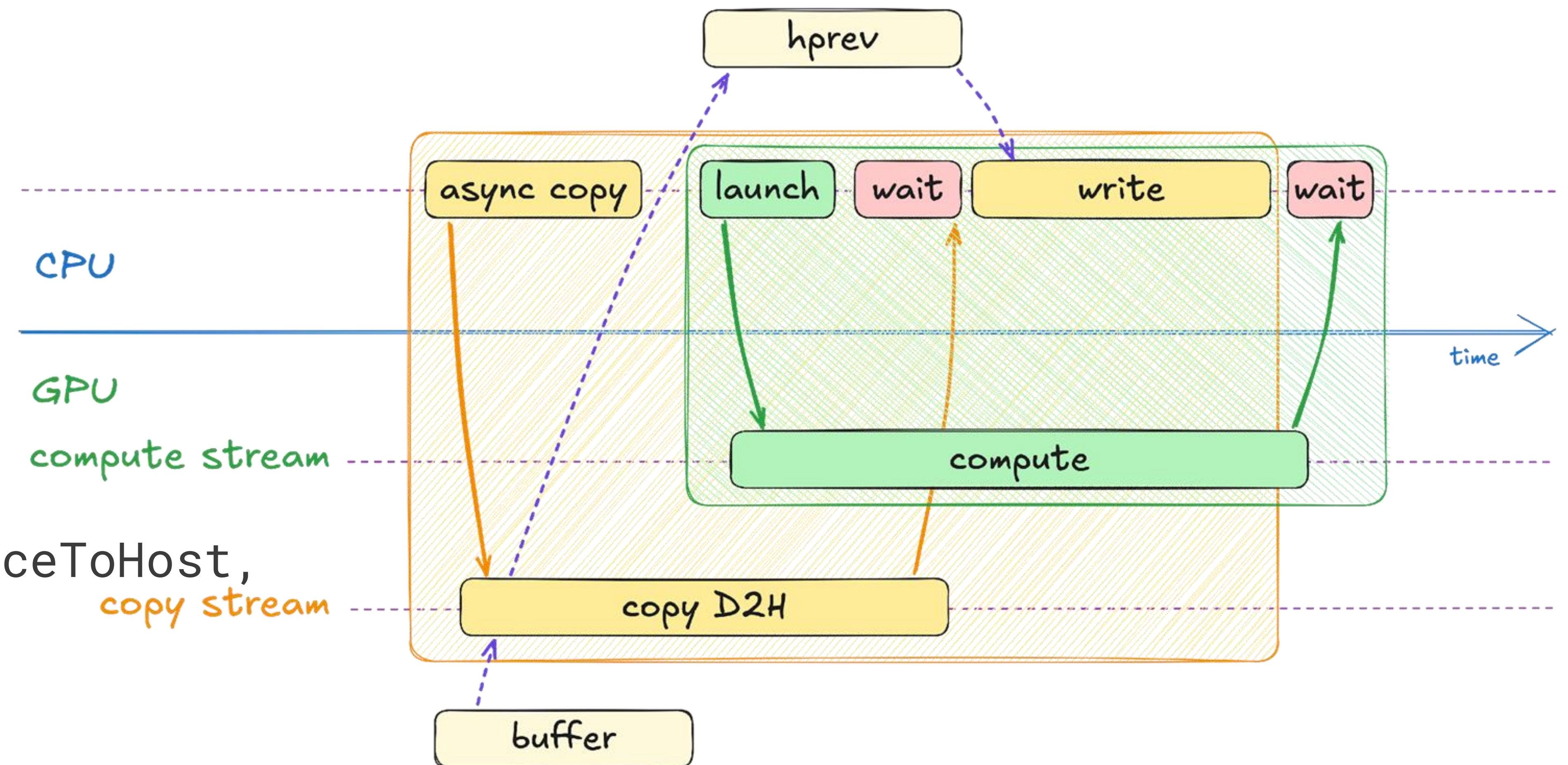
```
cudaMemcpyAsync(hprev_ptr,  
               dprev_ptr,  
               num_cells * sizeof(float),  
               cudaMemcpyDeviceToHost,  
               copy_stream);
```

```
for (int step = 0; step < steps; step++)  
{  
    simulate(width, height, dprev, dnext, compute_stream);  
    dprev.swap(dnext);  
}
```

```
cudaStreamSynchronize(copy_stream);
```

호스트 벡터를 읽기 전에 복사 스트림 동기화

```
store(write_step, height, width, hprev);  
cudaStreamSynchronize(compute_stream);
```



CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);
```

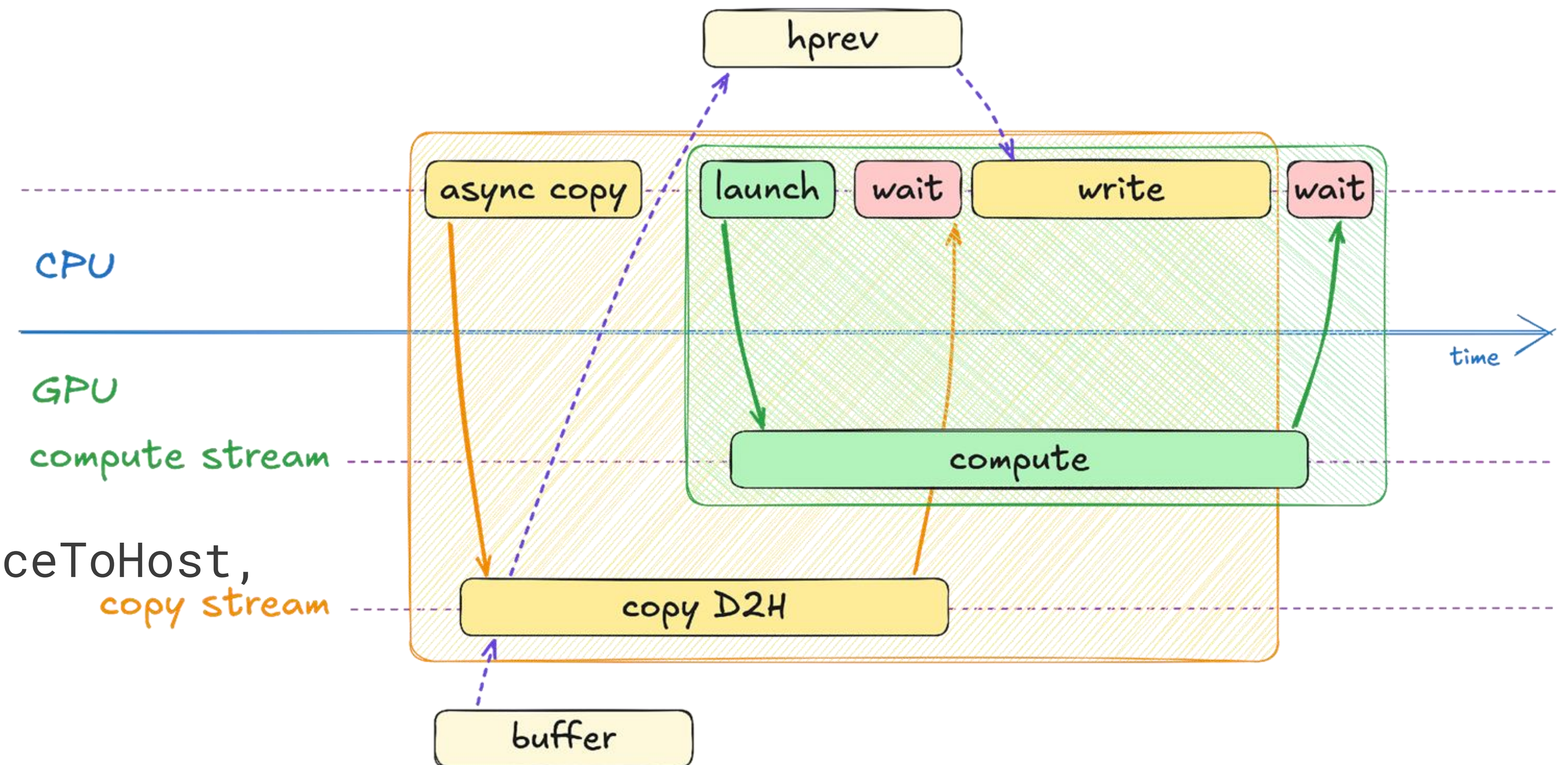
```
// ...
```

```
cudaMemcpyAsync(hprev_ptr,
                dprev_ptr,
                num_cells * sizeof(float),
                cudaMemcpyDeviceToHost,
                copy_stream);
```

```
for (int step = 0; step < steps; step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}
```

```
cudaStreamSynchronize(copy_stream);
```

```
store(write_step, height, width, hprev);
cudaStreamSynchronize(compute_stream);
```



다음 단계를 시작하기 전에 컴퓨트 스트림 동기화

CUDA 스트림

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

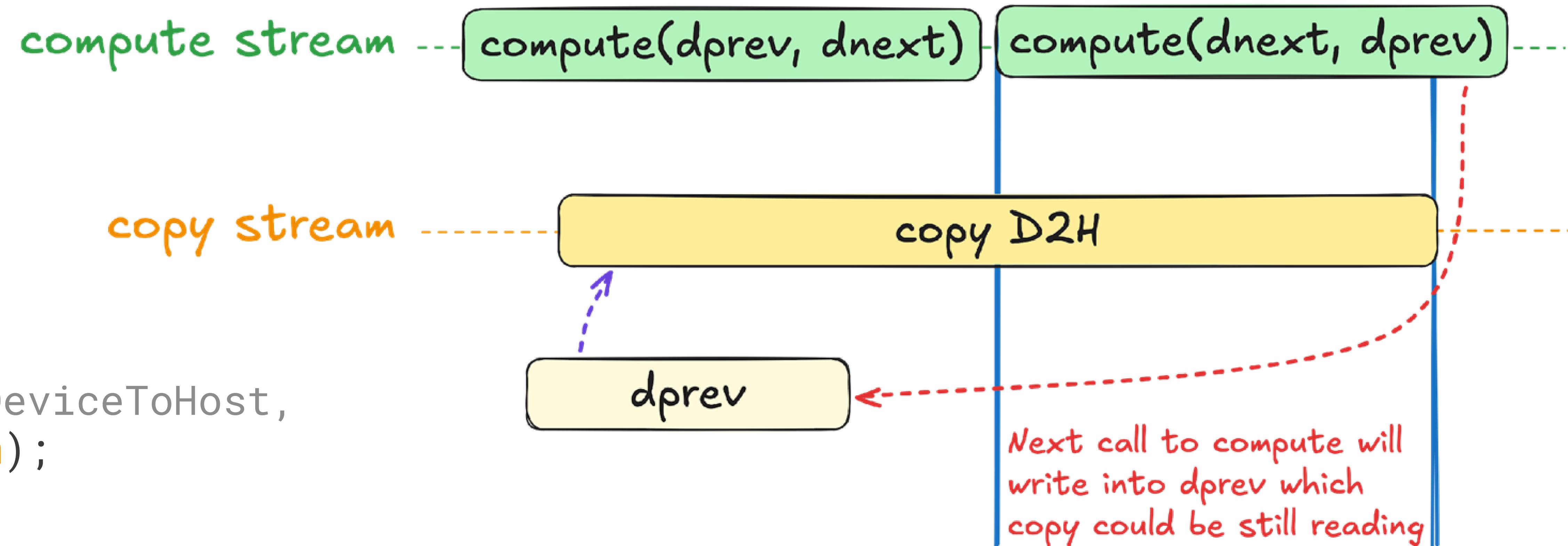
// ...

cudaMemcpyAsync(hprev_ptr,
                dprev_ptr,
                num_cells * sizeof(float),
                cudaMemcpyDeviceToHost,
                copy_stream);

for (int step = 0; step < steps; step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);

store(write_step, height, width, hprev);
cudaStreamSynchronize(compute_stream);
```



- 이 코드에는 데이터 경합(**data race**)이 있습니다!
- 다른 스트림은 순서대로 작업을 실행합니다

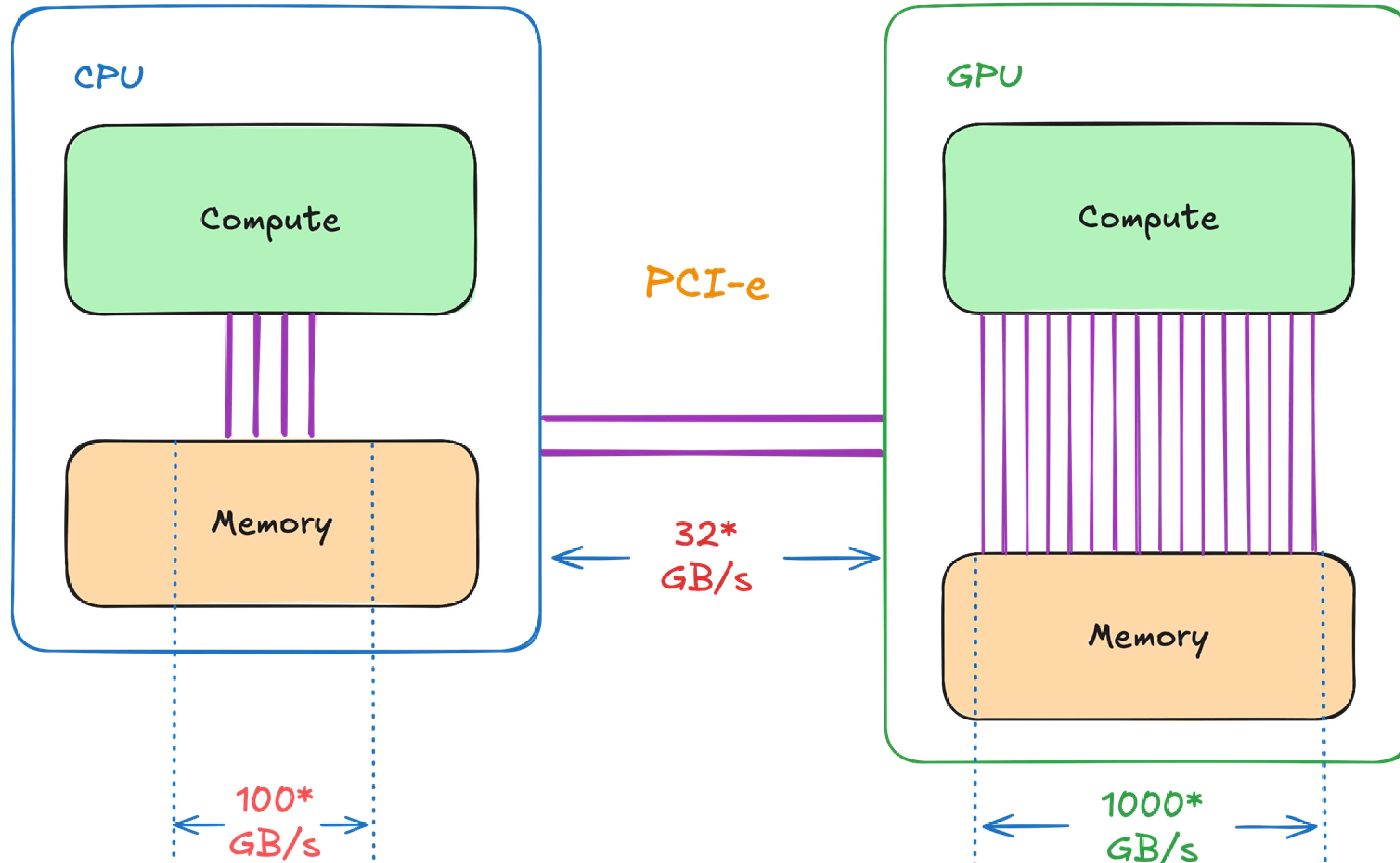
CUDA 스트림

```
thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());
cudaMemcpyAsync(htemp_ptr, dbuffer_ptr, num_bytes, cudaMemcpyDeviceToHost, copy_stream);

for (int step = 0; step < steps; step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}
```

- 대부분의 문제는 다른 수준의 간접 참조로 해결할 수 있습니다.
- 다음을 수행해 봅시다:
 - 디바이스 버퍼 할당
 - 컴퓨트 스트림에서 dprev를 복사합니다.
 - 이 버퍼에서 hprev로 비동기 복사 시작

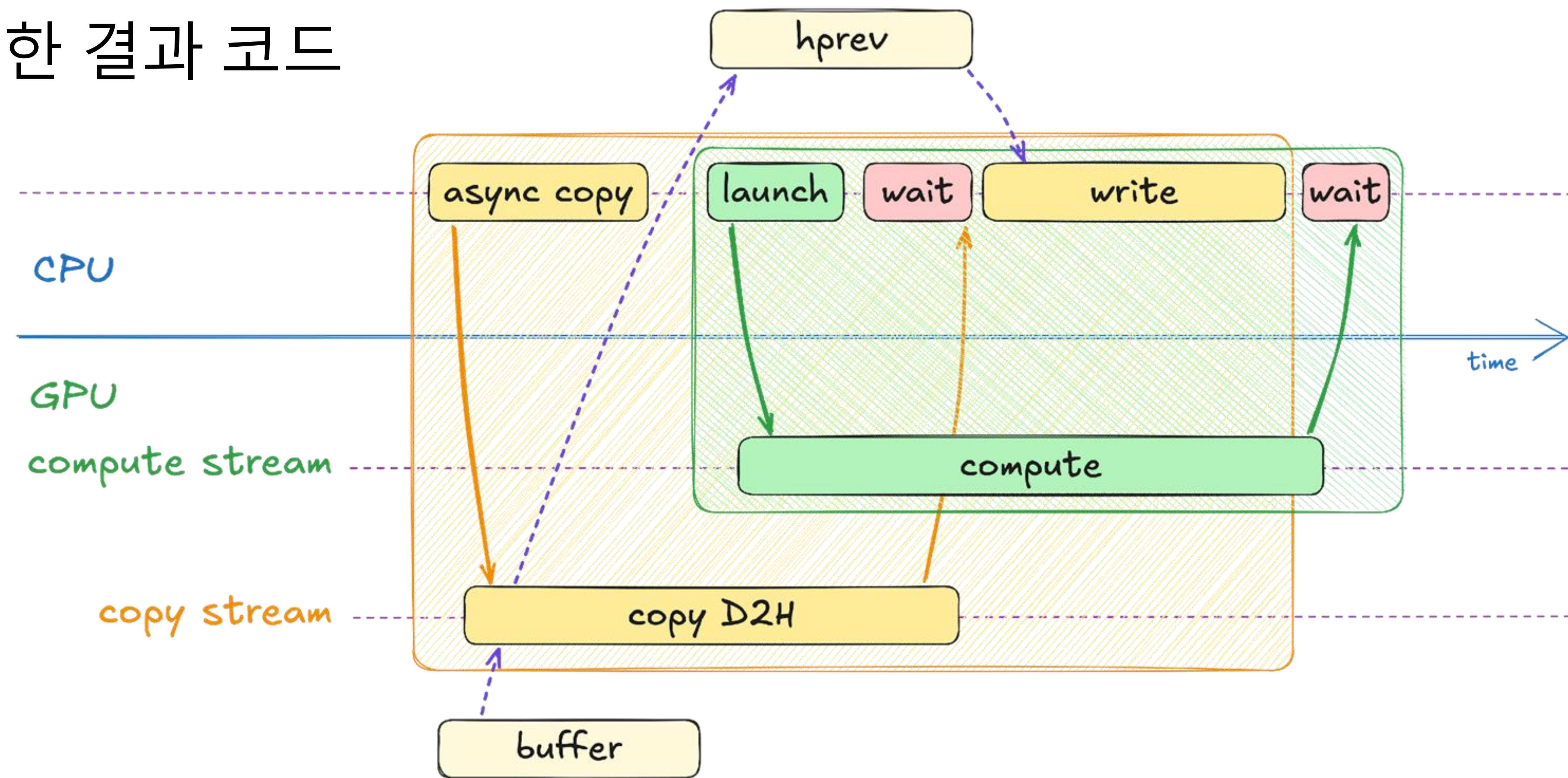
D2D 복사 오버헤드 평가



연습: 중복 계산 및 복사

15분

- `thrust::copy` 대신 `cudaMemcpyAsync` 사용
- 다른 스트림에 컴퓨트 넣기 및 비동기 복사
- Nsight Systems를 사용한 결과 코드



02.03-Streams/02.03.02-Exercise-Async-Copy.ipynb

연습: 중복 계산 및 복사

솔루션

```
cudaStream_t copy_stream, compute_stream;  
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

compute_stream 및 copy_stream 생성

```
thrust::host_vector<float> hprev(height * width);  
  
thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());  
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),  
                thrust::raw_pointer_cast(dbuffer.data()),  
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,  
                copy_stream);  
  
for (int compute_step = 0; compute_step < compute_steps; compute_step++)  
{  
    simulate(width, height, dprev, dnext, compute_stream);  
    dprev.swap(dnext);  
}  
  
cudaStreamSynchronize(copy_stream);  
store(write_step, height, width, hprev);  
  
cudaStreamSynchronize(compute_stream);
```


연습: 중복 계산 및 복사

솔루션

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

thrust::host_vector<float> hprev(height * width);

thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),
                thrust::raw_pointer_cast(dbuffer.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,
                copy_stream);

for (int compute_step = 0; compute_step < compute_steps; compute_step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);
store(write_step, height, width, hprev);

cudaStreamSynchronize(compute_stream);
```

스테이징 버퍼에 동기적으로
복사

연습: 중복 계산 및 복사

솔루션

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

thrust::host_vector<float> hprev(height * width);

thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),
                thrust::raw_pointer_cast(dbuffer.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,
                copy_stream);

for (int compute_step = 0; compute_step < compute_steps; compute_step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);
store(write_step, height, width, hprev);

cudaStreamSynchronize(compute_stream);
```

copy stream 스테이징 버퍼에서 **copy stream** 의 호스트 벡터로 비동기적으로 복사

연습: 중복 계산 및 복사

솔루션

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

thrust::host_vector<float> hprev(height * width);

thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),
                thrust::raw_pointer_cast(dbuffer.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,
                copy_stream);

for (int compute_step = 0; compute_step < compute_steps; compute_step++)
{
    simulate(width, height, dprev, dnext, compute_stream); | compute stream에서 컴퓨트 연산 시작
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);
store(write_step, height, width, hprev);

cudaStreamSynchronize(compute_stream);
```


연습: 중복 계산 및 복사

솔루션

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

thrust::host_vector<float> hprev(height * width);

thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin());
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),
                thrust::raw_pointer_cast(dbuffer.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,
                copy_stream);

for (int compute_step = 0; compute_step < compute_steps; compute_step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

cudaStreamSynchronize(copy_stream);
store(write_step, height, width, hprev);

cudaStreamSynchronize(compute_stream);
```

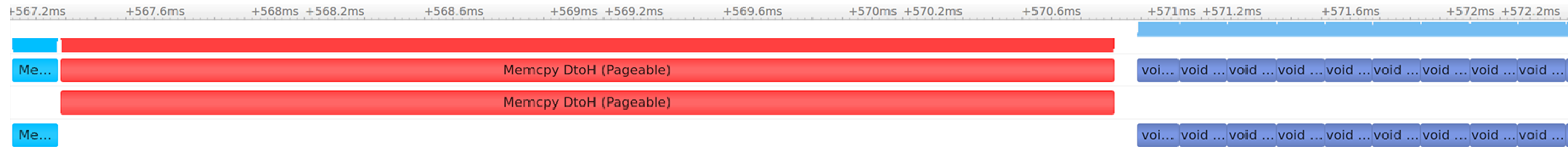
데이터를 읽기 전에 **copy stream** 의 복사
가 완료될 때까지 기다리십시오.

CUDA 스트림

D2D

D2H

GPU



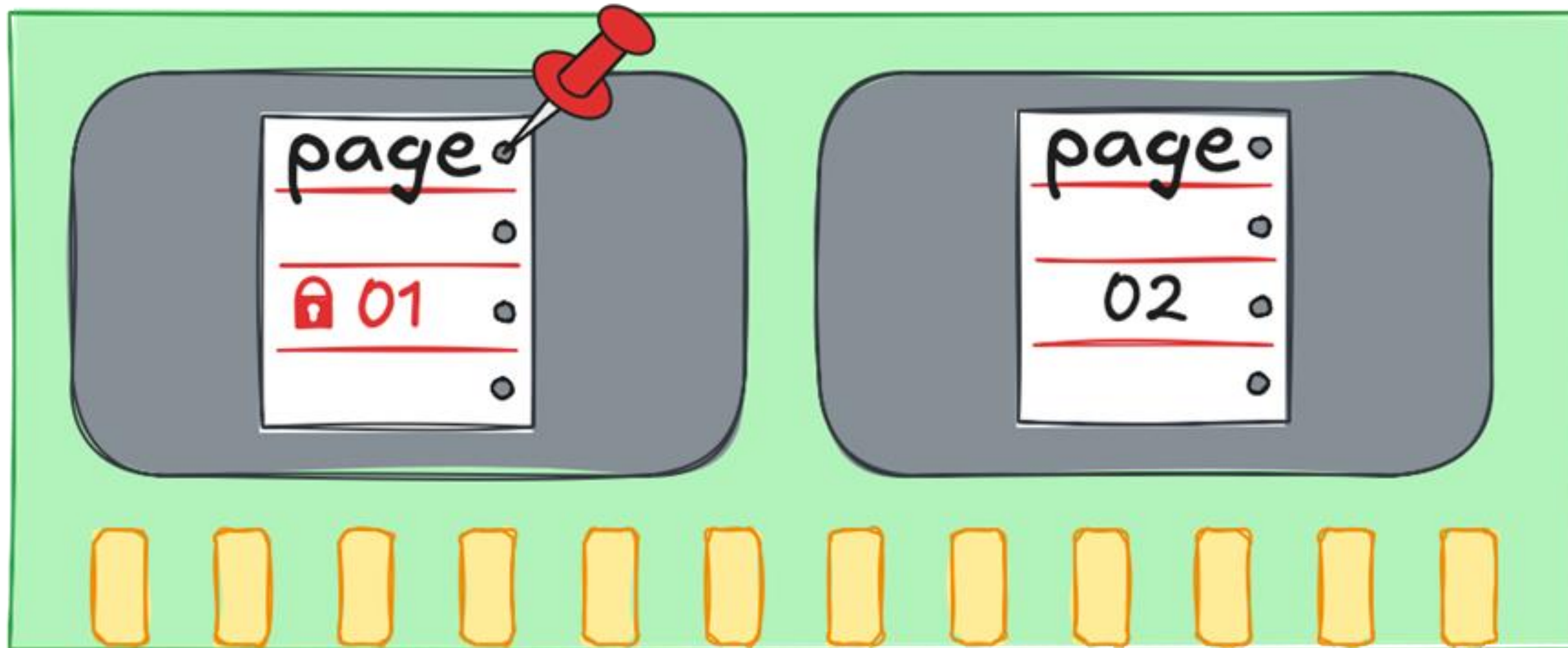
CPU



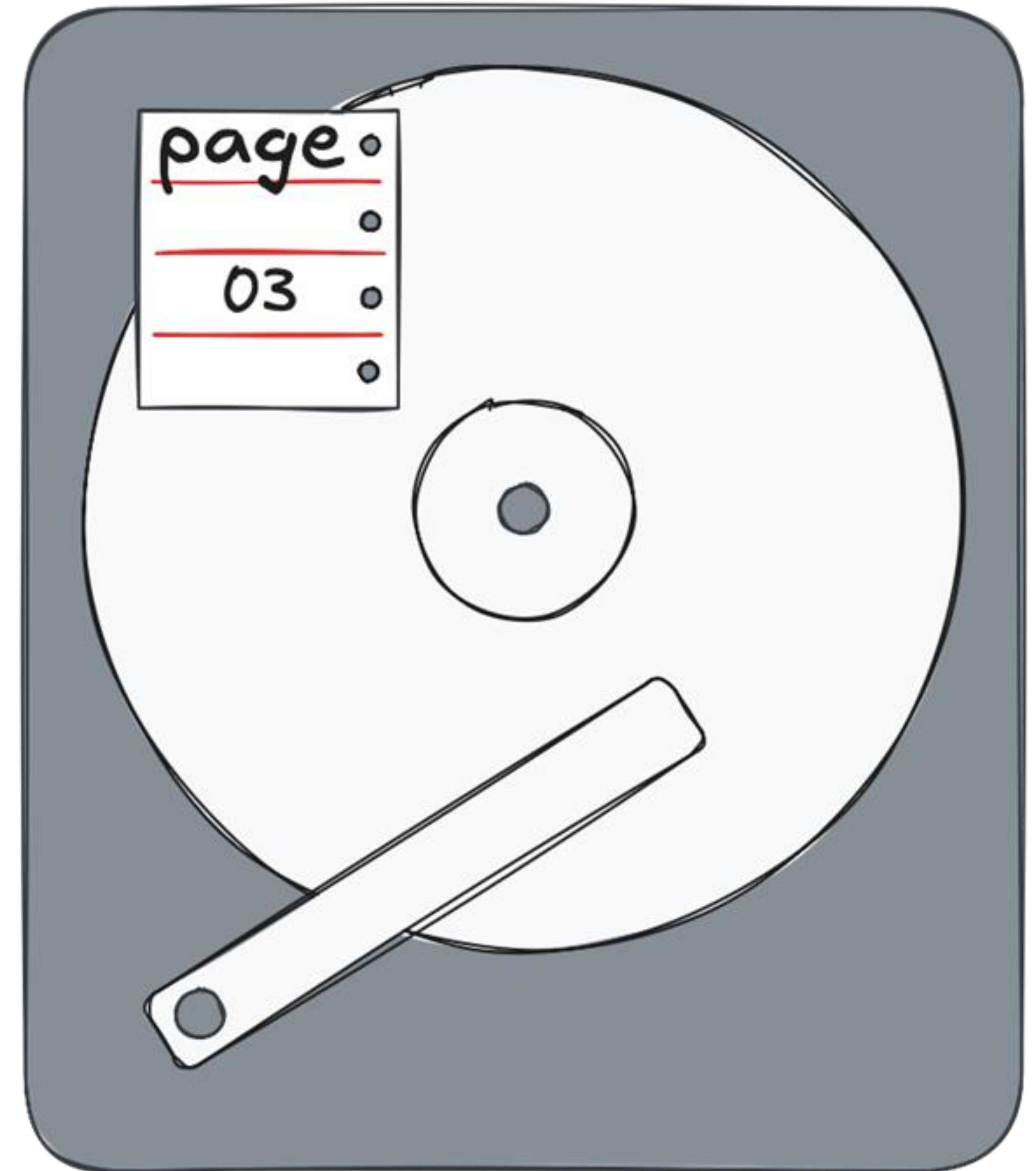
- D2D 복사는 D2H에 비해 오버헤드가 거의 없습니다.
- 그럼에도 불구하고 프로파일링에 문제가 있습니다.
- 복사가 완료되기 전에 변환이 시작되지 않음

가상 메모리 개요

RAM

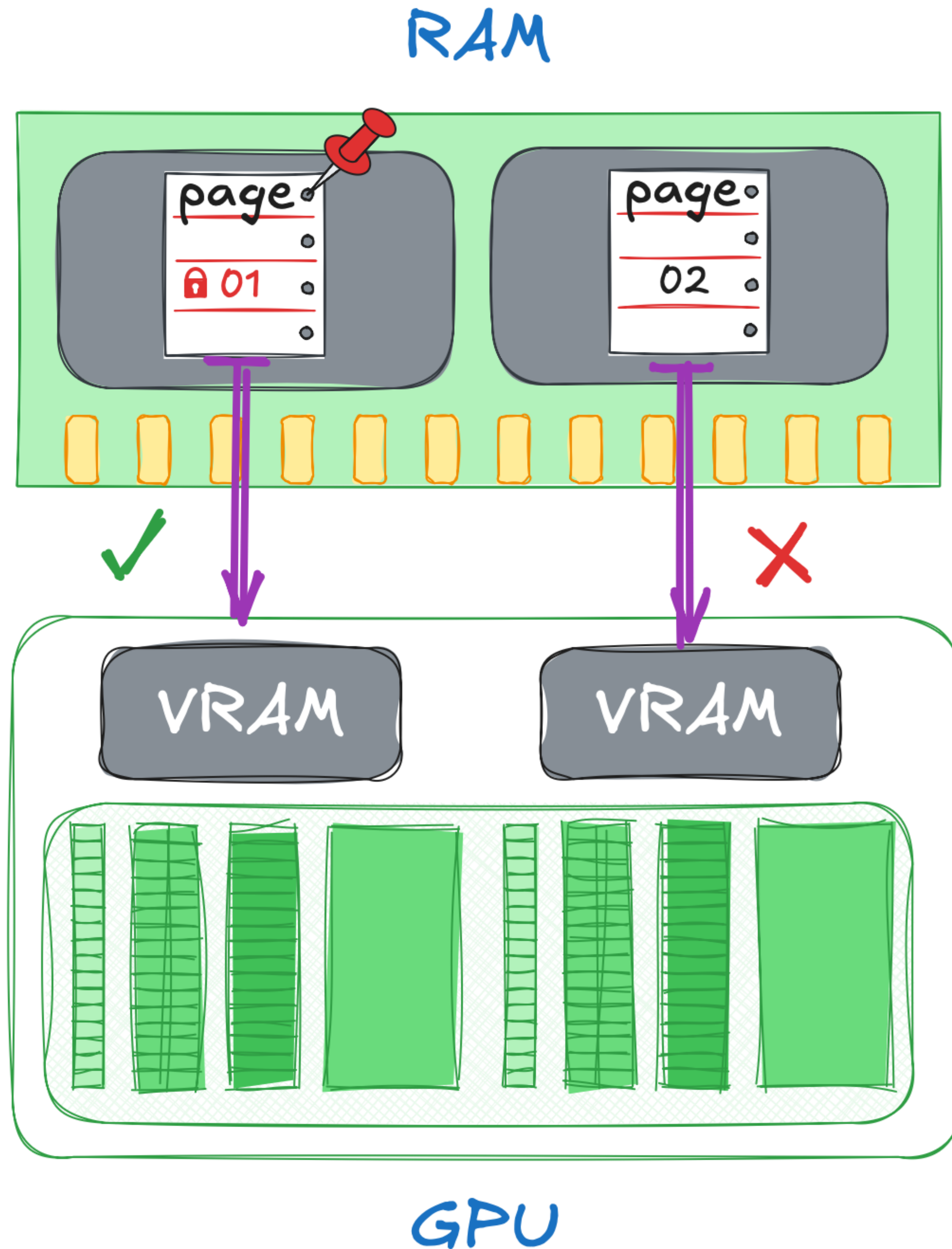


Disk



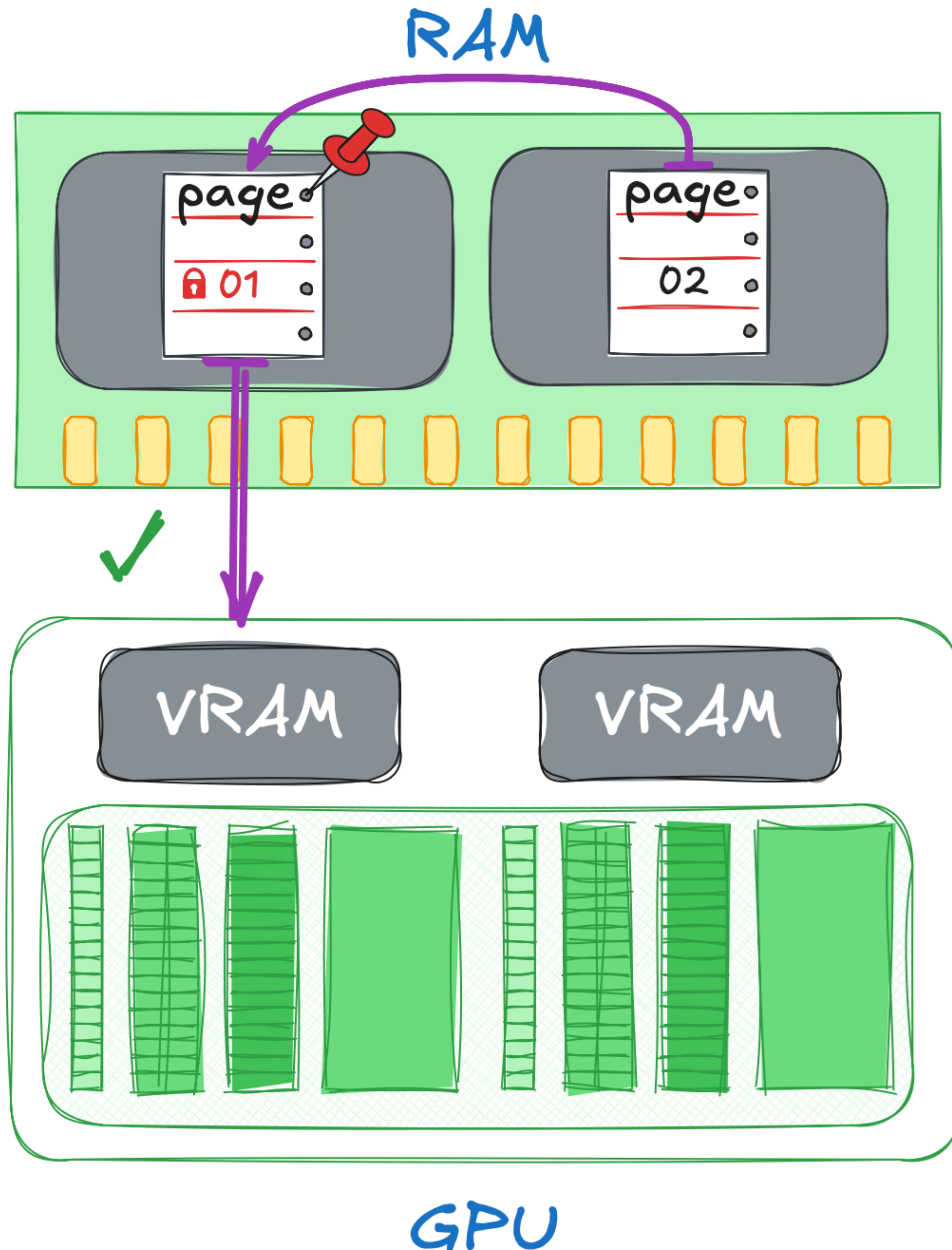
- Virtual memory consists of pages
- Any page can end up on disk, unless it was locked or **pinned**

가상 메모리 개요



- GPU는 Pinned 메모리에서만 읽을 수 있습니다!
- 그러나 `cudaMemcpyAsync` 어떻게 작동합니까?

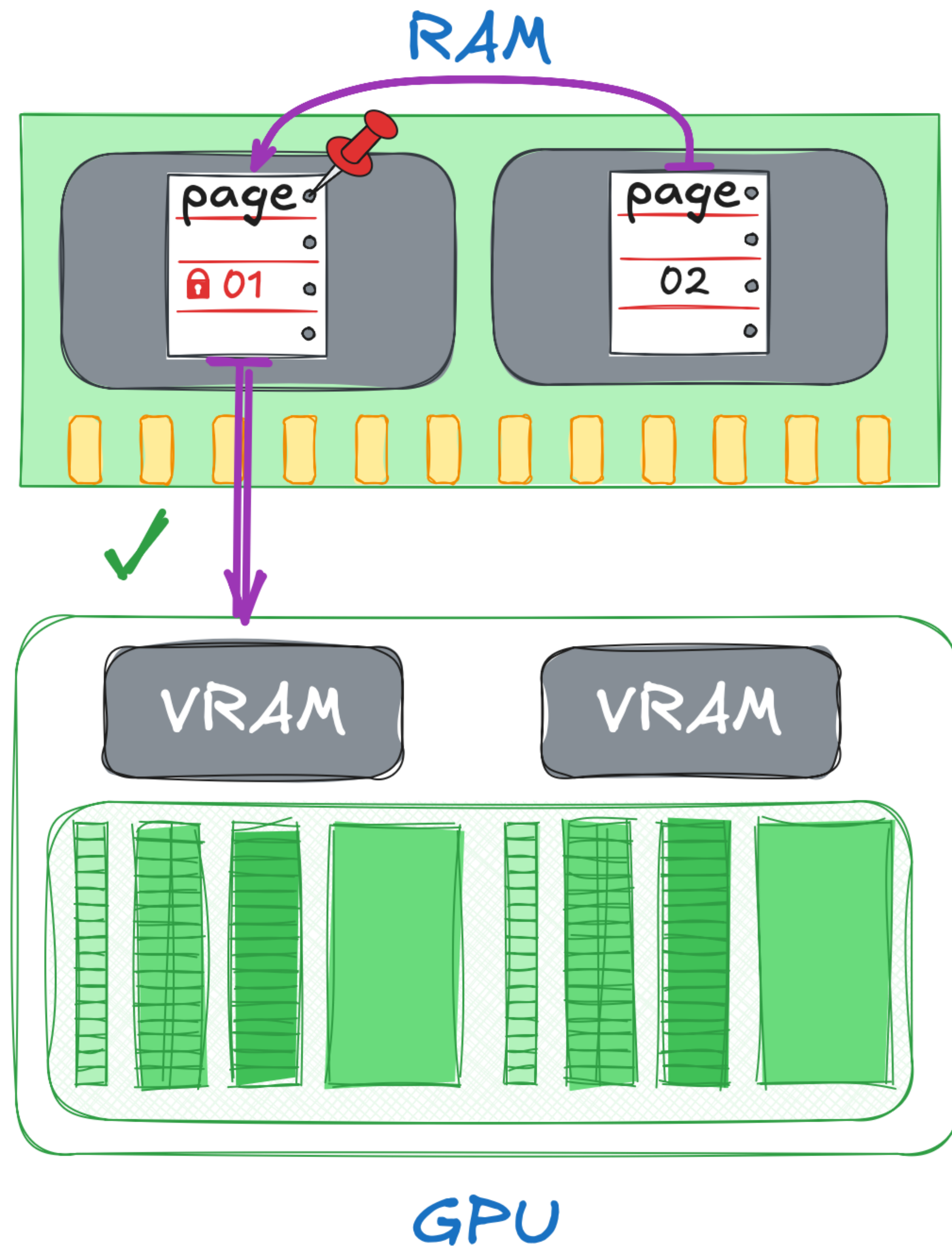
가상 메모리 개요



- Pageable 메모리에서 Pinned 메모리의 스테이징 버퍼를 통해 전송됩니다.
- 따라서 Pageable 메모리에서 전송하는 데 시간이 더 오래 걸리고 비동기성을 방해합니다.
- 다행히도 Pinned 메모리를 다음과 같이 사용할 수 있습니다.
- `thrust::universal_host_pinned_vector`

연습: Pinned 메모리 사용

5분



- `thrust::universal_host_pinned_vector`를 사용하여 Pinned 메모리에 호스트 벡터를 할당합니다
- 변경 후 애플리케이션 프로파일링
- 디바이스-투-호스트 복사본을 찾아 컴퓨터 연산과 겹치는지 확인합니다.

연습: Pinned 메모리 사용

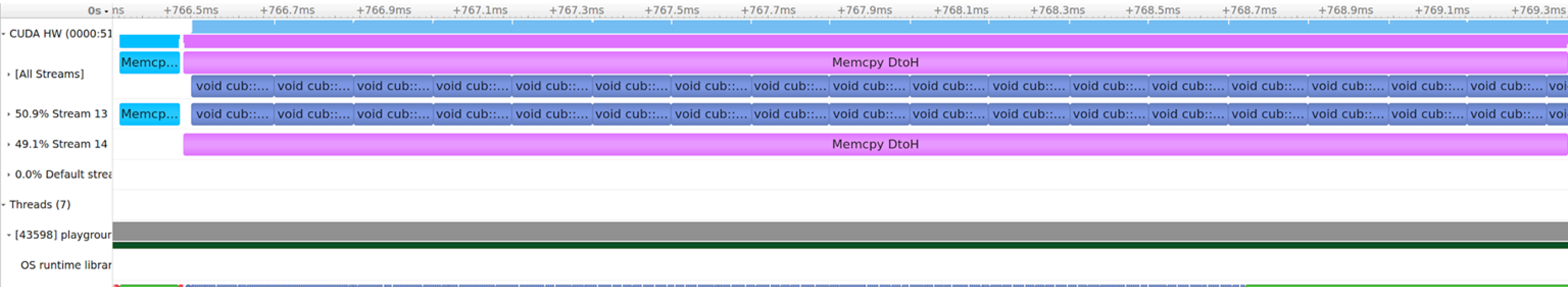
솔루션

```
cudaStream_t copy_stream, compute_stream;  
cudaStreamCreate(&compute_stream);  
cudaStreamCreate(&copy_stream);
```

```
thrust::universal_host_pinned_vector<float> hprev(height * width);
```

```
cudaMemcpy(thrust::raw_pointer_cast(dbuffer.data()),  
           thrust::raw_pointer_cast(dprev.data()),  
           num_cells * sizeof(float), cudaMemcpyDeviceToDevice);
```

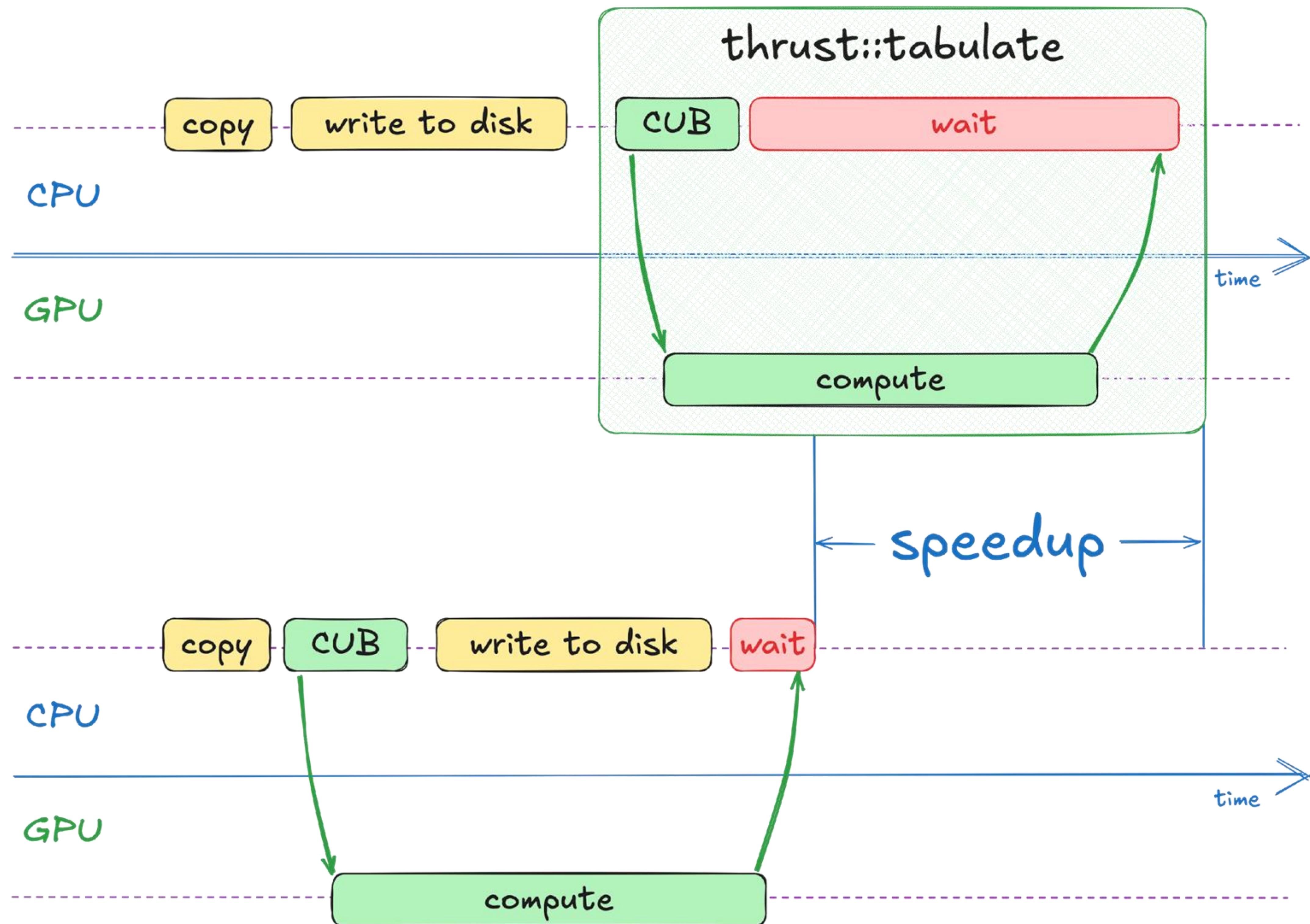
...



핵심 요약

비동기 프로그래밍 모델 선호 조건:

- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산



핵심 요약

비동기 프로그래밍 모델 선호 조건:

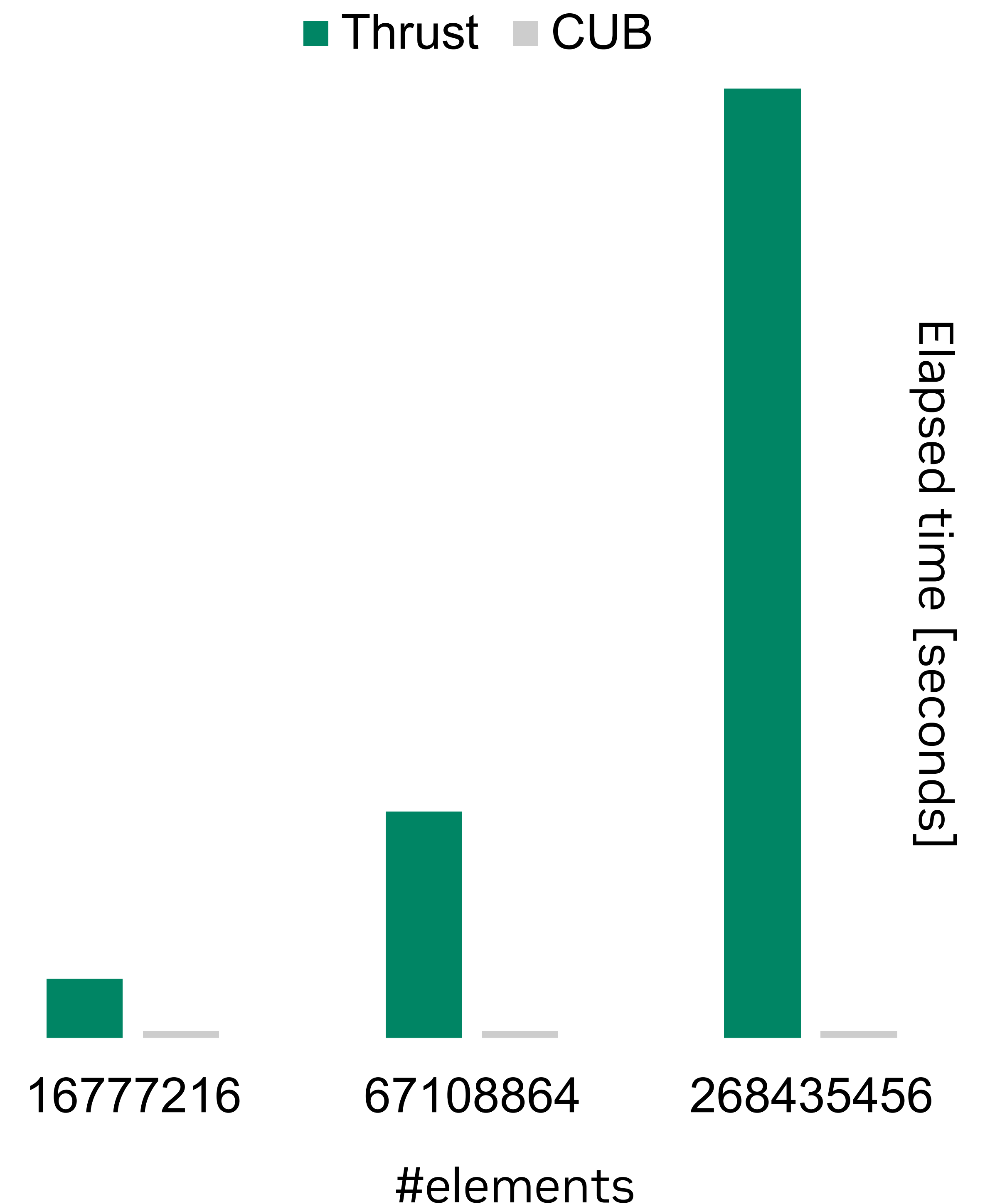
- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산

CUB 사용 조건

- 비동기 범용 병렬 알고리즘

CUB

```
auto begin = std::chrono::high_resolution_clock::now();  
auto cell_ids = thrust::make_counting_iterator(0);  
cub::DeviceTransform::Transform(  
    cell_ids, out.begin(), num_cells, compute);  
auto end = std::chrono::high_resolution_clock::now();
```



핵심 요약

비동기 프로그래밍 모델 선호 조건:

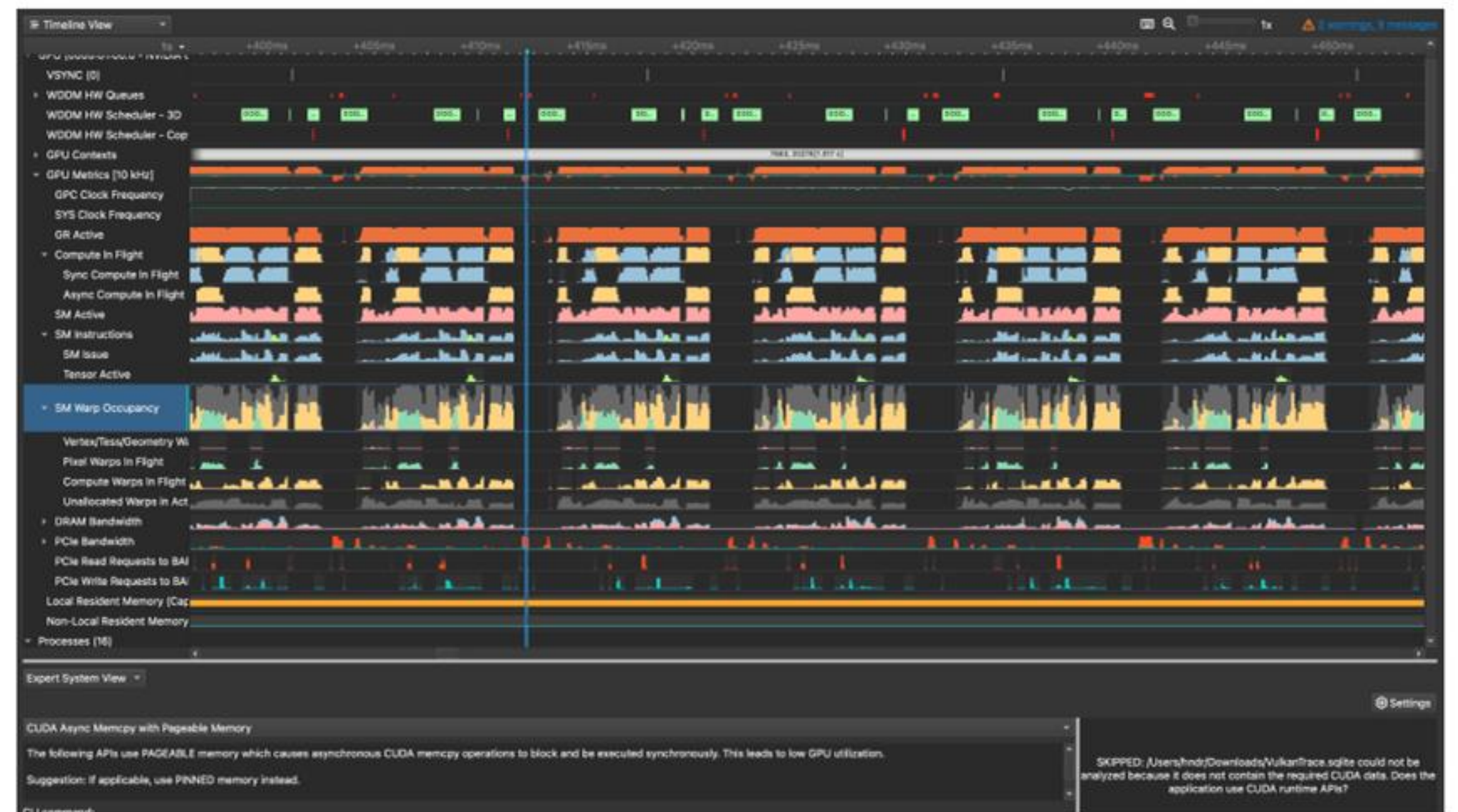
- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산

CUB 사용 조건

- 비동기 범용 병렬 알고리즘

NVIDIA Nsight 시스템 사용 조건:

- 코드를 프로파일링하려면
- 최적화 기회 식별
- 비동기 실행 시각화
- NVTX를 사용하여 프로파일링 간소화



핵심 요약

비동기 프로그래밍 모델 선호 조건:

- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산

CUB 사용 조건

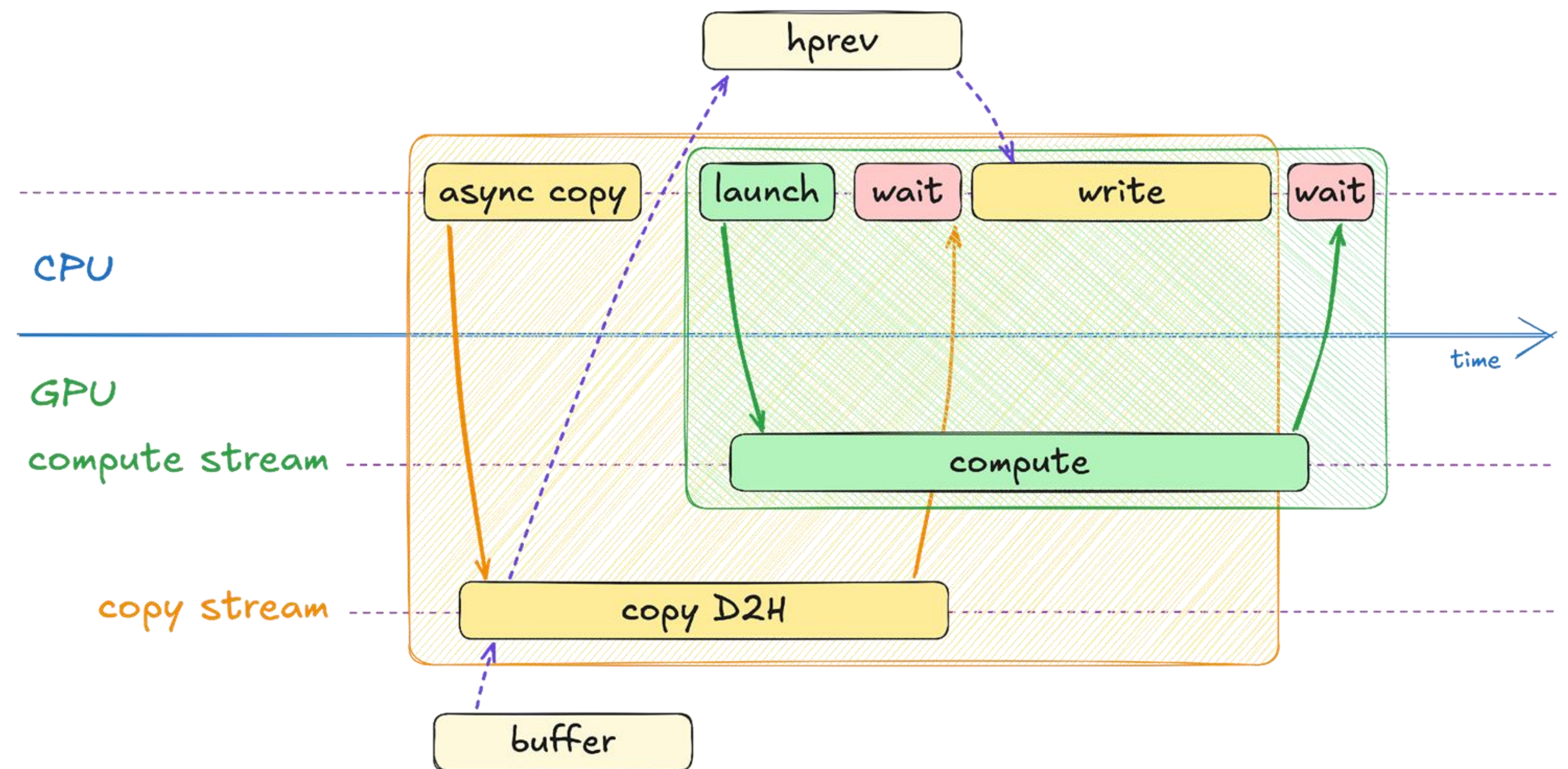
- 비동기 범용 병렬 알고리즘

NVIDIA Nsight 시스템 사용 조건:

- 코드를 프로파일링하려면
- 최적화 기회 식별
- 비동기 실행 시각화
- NVTX를 사용하여 프로파일링 간소화

cudaStream_t 사용 조건:

- CPU 및 기타 스트림과의 비동기 실행을 활성화하려면



핵심 요약

비동기 프로그래밍 모델 선호 조건:

- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산

CUB 사용 조건

- 비동기 범용 병렬 알고리즘

NVIDIA Nsight 시스템 사용 조건:

- 코드를 프로파일링하려면
- 최적화 기회 식별
- 비동기 실행 시각화
- NVTX를 사용하여 프로파일링 간소화

cudaStream_t 사용 조건:

- CPU 및 기타 스트림과의 비동기 실행을 활성화하려면

cudaMemcpyAsync 사용 조건:

- CPU에 대해 메모리 복사본을 비동기적으로 만듭니다

```
cudaError_t cudaMemcpyAsync(
```

```
void*          dst, —————> Destination memory address
```

```
const void*    src, —————> Source memory address
```

```
size_t         count, —————> Size in bytes to copy
```

```
cudaMemcpyKind kind —————> cudaMemcpyHostToHost
```

```
);
```

```
...
```


핵심 요약

비동기 프로그래밍 모델 선호 조건:

- 메모리 전송을 겹치려면
- CPU 및 GPU 계산을 겹치기 위해
- 또는 다른 GPU 계산

CUB 사용 조건

- 비동기 범용 병렬 알고리즘

NVIDIA Nsight 시스템 사용 조건:

- 코드를 프로파일링하려면
- 최적화 기회 식별
- 비동기 실행 시각화
- NVTX를 사용하여 프로파일링 간소화

cudaStream_t 사용 조건:

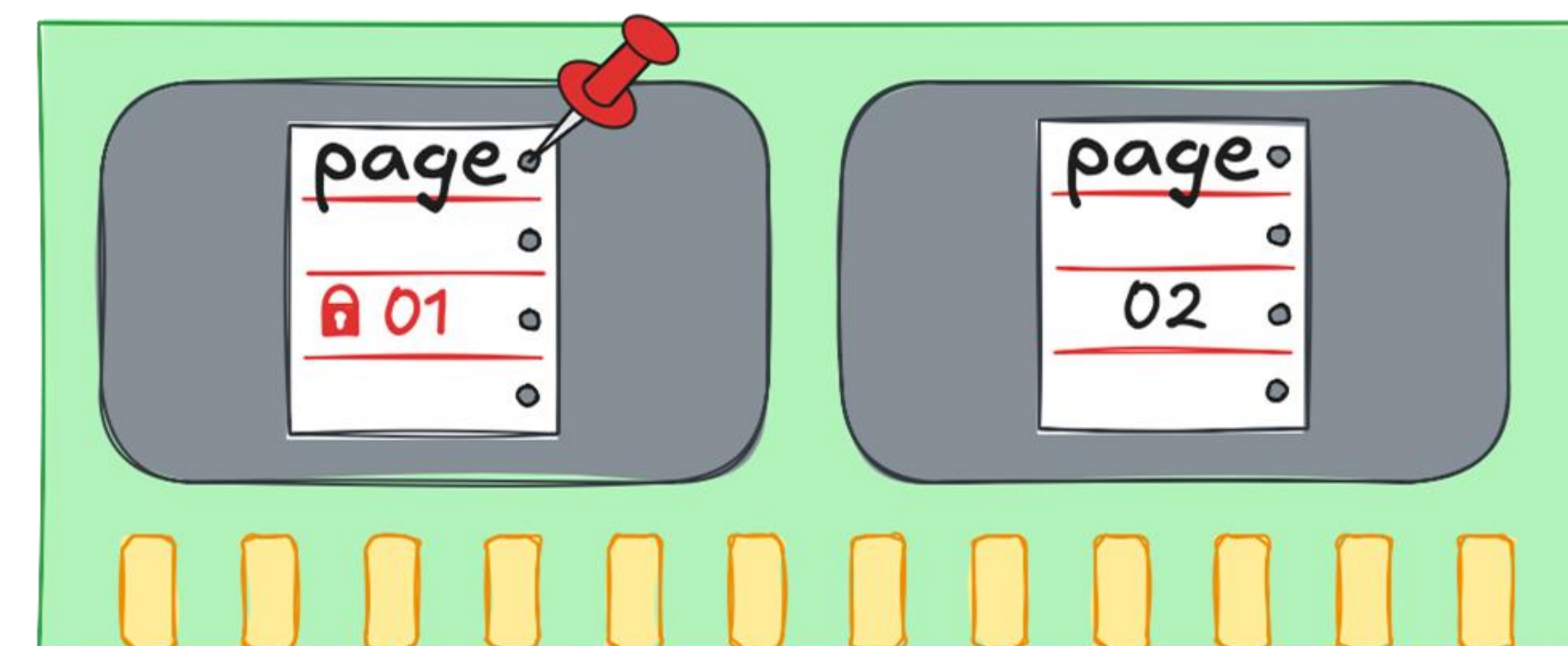
- CPU 및 기타 스트림과의 비동기 실행을 활성화하려면

cudaMemcpyAsync 사용 조건:

- CPU에 대해 메모리 복사본을 비동기적으로 만듭니다

Pinned 메모리 사용 조건:

- 디바이스에서 비동기적으로 복사하려면
- 디바이스에서 더 빠르게 복사



휴식
10 분